

Escape From Heidelberg Dynamics Godot

Generated by Doxygen 1.11.0

1 Game Systems, tested using Xunit	1
1.1 Overview	1
1.2 Setting Up the Environment	1
1.2.1 Using Nix	1
1.2.2 On Ubuntu	1
1.2.3 On Windows	1
1.2.4 Running Tests	2
1.3 Coding Style	2
1.4 Health.cs	2
2 Game Systems, tested using Xunit	3
2.1 Overview	3
2.2 Setting Up the Environment	3
2.2.1 Using Nix	3
2.2.2 On Ubuntu	3
2.2.3 On Windows	3
2.2.4 Running Tests	3
2.3 Coding Style	4
2.4 Health.cs	4
3 Namespace Index	4
3.1 Namespace List	4
4 Hierarchical Index	5
4.1 Class Hierarchy	5
5 Class Index	5
5.1 Class List	5
6 File Index	6
6.1 File List	6
7 Namespace Documentation	7
7.1 TestProject1 Namespace Reference	7
8 Class Documentation	7
8.1 BossHealth Class Reference	7
8.1.1 Detailed Description	9
8.1.2 Member Data Documentation	9
8.2 Bossturret Class Reference	9
8.2.1 Detailed Description	11
8.2.2 Member Function Documentation	11
8.2.3 Member Data Documentation	12
8.3 explosion Class Reference	14
8.3.1 Detailed Description	15

8.3.2 Member Function Documentation	15
8.4 Health Class Reference	15
8.4.1 Detailed Description	18
8.4.2 Constructor & Destructor Documentation	18
8.4.3 Member Function Documentation	19
8.4.4 Member Data Documentation	24
8.4.5 Property Documentation	25
8.5 InteractionArea Class Reference	27
8.5.1 Detailed Description	29
8.5.2 Member Function Documentation	29
8.5.3 Member Data Documentation	32
8.6 Level_4 Class Reference	33
8.6.1 Detailed Description	34
8.6.2 Member Function Documentation	34
8.7 Player Class Reference	34
8.7.1 Detailed Description	38
8.7.2 Member Function Documentation	38
8.7.3 Member Data Documentation	42
8.8 PlayerHealth Class Reference	45
8.8.1 Detailed Description	47
8.8.2 Member Data Documentation	47
8.9 Projectile Class Reference	47
8.9.1 Detailed Description	48
8.9.2 Member Function Documentation	49
8.9.3 Member Data Documentation	50
8.10 Turret Class Reference	51
8.10.1 Detailed Description	53
8.10.2 Member Function Documentation	53
8.10.3 Member Data Documentation	54
8.11 TurretHealth Class Reference	55
8.11.1 Detailed Description	57
8.11.2 Member Data Documentation	57
8.12 TestProject1.UnitTest1 Class Reference	57
8.12.1 Detailed Description	59
8.12.2 Constructor & Destructor Documentation	59
8.12.3 Member Function Documentation	60
8.12.4 Member Data Documentation	63
9 File Documentation	63
9.1 BossHealth.cs	63
9.2 Bossturret.cs	64
9.3 Player.cs	64

9.4 PlayerHealth.cs	68
9.5 turret.cs	68
9.6 TurretHealth.cs	69
9.7 GlobalUsings.cs	69
9.8 Health.cs	69
9.9 Health.cs	70
9.10 UnitTest1.cs	72
9.11 UnitTest1.cs	73
9.12 InteractionArea.cs	74
9.13 level_4.cs	75
9.14 explosion.cs	76
9.15 projectile.cs	76
Index	79

1 Game Systems, tested using Xunit

1.1 Overview

This project includes a basic health system implemented in C# with tests written using XUnit.

1.2 Setting Up the Environment

1.2.1 Using Nix

1. **Install Nix:** Follow the instructions at <https://nixos.org/download.html>.
2. **Enter Nix Shell:** Run the following command in the project directory to enter the Nix environment with Godot 4 and .NET SDK 8.

```
nix-shell
```

1.2.2 On Ubuntu

If you prefer to set up .NET SDK 8 directly on Ubuntu without using Nix:

1. **Install the Microsoft package repository and GPG keys:**

```
wget https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
sudo apt-get install -y dotnet-sdk-8.0
```

1.2.3 On Windows

Install Nix on WSL.

1.2.4 Running Tests

To run the XUnit tests in the `TestProject1` directory, use the following command:

```
dotnet test TestProject1.sln
```

This will build the project and execute all tests, providing a summary of the test results.

This setup and guide are intended for internal use to streamline the development and testing process for the game systems project.

1.3 Coding Style

The Coding Style is based on the MS .NET C# coding style:

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Class Names and Namespaces are named in PascalCase.

Functions, Parameters, and Variables of any kind are to be named in camelCase.

Both public and private variables are to be prefixed with an underscore.

Example:

```
using System;

public class Health
{
    private int _hp;

    // Constructor to initialize health with a starting value
    public Health(int initialHp)
    {
        _hp = initialHp;
    }

    // Function to apply damage, using Math.Max to ensure hp doesn't go below 0
    public void Damage(int amount)
    {
        _hp = Math.Max(0, _hp - amount);
    }

    // Function to apply healing, using Math.Min to ensure hp doesn't exceed a maximum value (e.g., 100)
    public void Heal(int amount)
    {
        int maxHealth = 100; // You can modify this value based on your requirements
        _hp = Math.Min(maxHealth, _hp + amount);
    }

    // Function to get the current health value
    public int GetHealth()
    {
        return _hp;
    }
}
```

1.4 Health.cs

Health.cs contains the health system.

Entities have a separate `Health` and `Shield`, the shield gets depleted and regenerated more easily than health, and once health drops to zero the entity dies, even if it has shield points left.

All calculations are to be made in integer form for speed, shifted one digit to the left. e.g. 852 equals 85.2f

2 Game Systems, tested using XUnit

2.1 Overview

This project includes a basic health system implemented in C# with tests written using XUnit.

2.2 Setting Up the Environment

2.2.1 Using Nix

1. **Install Nix:** Follow the instructions at <https://nixos.org/download.html>.
2. **Enter Nix Shell:** Run the following command in the project directory to enter the Nix environment with Godot 4 and .NET SDK 8.
`nix-shell`

2.2.2 On Ubuntu

If you prefer to set up .NET SDK 8 directly on Ubuntu without using Nix:

1. **Install the Microsoft package repository and GPG keys:**

```
wget https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
sudo apt-get install -y dotnet-sdk-8.0
```

2.2.3 On Windows

Install Nix on WSL.

2.2.4 Running Tests

To run the XUnit tests in the `TestProject1` directory, use the following command:
`dotnet test TestProject1.sln`

This will build the project and execute all tests, providing a summary of the test results.

This setup and guide are intended for internal use to streamline the development and testing process for the game systems project.

2.3 Coding Style

The Coding Style is based on the MS .NET C# coding style:

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Class Names and Namespaces are named in PascalCase.

Functions, Parameters, and Variables of any kind are to be named in camelCase.

Both public and private variables are to be prefixed with an underscore.

Example:

```
using System;

public class Health
{
    private int _hp;

    // Constructor to initialize health with a starting value
    public Health(int initialHp)
    {
        _hp = initialHp;
    }

    // Function to apply damage, using Math.Max to ensure hp doesn't go below 0
    public void Damage(int amount)
    {
        _hp = Math.Max(0, _hp - amount);
    }

    // Function to apply healing, using Math.Min to ensure hp doesn't exceed a maximum value (e.g., 100)
    public void Heal(int amount)
    {
        int maxHealth = 100; // You can modify this value based on your requirements
        _hp = Math.Min(maxHealth, _hp + amount);
    }

    // Function to get the current health value
    public int GetHealth()
    {
        return _hp;
    }
}
```

2.4 Health.cs

Health.cs contains the health system.

Entities have a separate [Health](#) and Shield, the shield gets depleted and regenerated more easily than health, and once health drops to zero the entity dies, even if it has shield points left.

All calculations are to be made in integer form for speed, shifted one digit to the left. e.g. 852 equals 85.2f

3 Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

[TestProject1](#)

7

4 Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AnimatedSprite2D	
explosion	14
Area2D	
InteractionArea	27
CharacterBody2D	
Bossturret	9
Player	34
Turret	51
Health	15
Node2D	
BossHealth	7
Level_4	33
PlayerHealth	45
TurretHealth	55
RigidBody2D	
Projectile	47
TestProject1.UnitTest1	57

5 Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BossHealth	
Represents the health of a boss turret	7
Bossturret	
Represents a boss turret	9
explosion	14
Health	
This class models the HP of an entity, health, shield, resistances, and vulnerabilities	15
InteractionArea	
Represents an interaction area that triggers events when the player enters it	27

Level_4		
Represents the logic for level 4 of the game		33
Player		
This class represents the player character in the game		34
PlayerHealth		
Represents the health of the player character		45
Projectile		
Represents a projectile entity in the game		47
Turret		
This class represents a turret in a 2D game		51
TurretHealth		
Represents the health of a turret in the game		55
TestProject1.UnitTest1		
Unit Tests for Health.cs		57

6 File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

InteractionArea.cs	74
characters/ BossHealth.cs	63
characters/ Bossturret.cs	64
characters/ Player.cs	64
characters/ PlayerHealth.cs	68
characters/ turret.cs	68
characters/ TurretHealth.cs	69
game-systems/TestProject1/ GlobalUsings.cs	69
game-systems/TestProject1/ Health.cs	69
game-systems/TestProject1/ UnitTest1.cs	72
levels/ level_4.cs	75
projectiles/ explosion.cs	76
projectiles/ projectile.cs	76
systems/TestProject1/ Health.cs	70
systems/TestProject1/ UnitTest1.cs	73

7 Namespace Documentation

7.1 TestProject1 Namespace Reference

Classes

- class [UnitTest1](#)

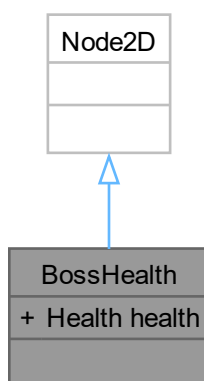
Unit Tests for Health.cs.

8 Class Documentation

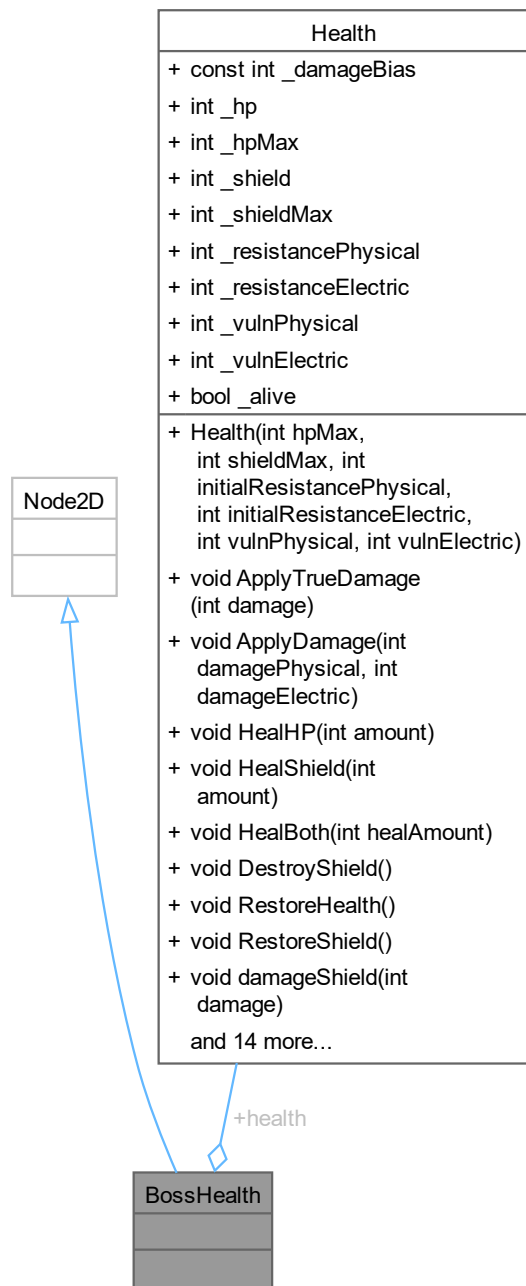
8.1 BossHealth Class Reference

Represents the health of a boss turret.

Inheritance diagram for BossHealth:



Collaboration diagram for BossHealth:



Public Attributes

- `Health health = new Health(1000, 1000, 100, 100, 100, 100)`

The boss's health information.

8.1.1 Detailed Description

Represents the health of a boss turret.

Definition at line 7 of file [BossHealth.cs](#).

8.1.2 Member Data Documentation

health

```
Health BossHealth.health = new Health(1000, 1000, 100, 100, 100, 100)
```

The boss's health information.

Definition at line 12 of file [BossHealth.cs](#).

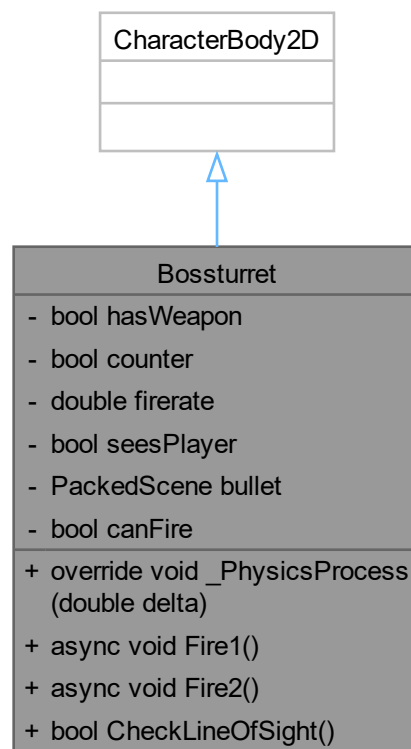
The documentation for this class was generated from the following file:

- [characters/BossHealth.cs](#)

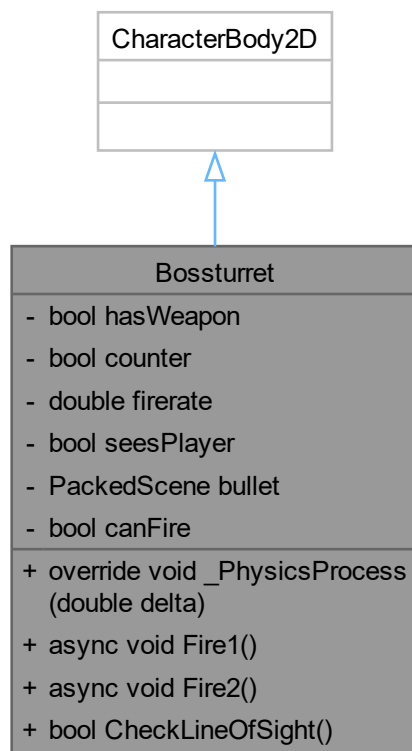
8.2 Bossturret Class Reference

Represents a boss turret.

Inheritance diagram for Bossturret:



Collaboration diagram for Bossturret:



Public Member Functions

- override void `_PhysicsProcess` (double delta)
Godot update function. Gets called every frame to update the turret's logic.
- async void `Fire1` ()
Fires the first bullet of the turret's weapon.
- async void `Fire2` ()
Fires the second bullet of the turret's weapon.
- bool `CheckLineOfSight` ()
Checks if the turret can see the player.

Private Attributes

- bool `hasWeapon` = true
Indicates whether the turret has a weapon.
- bool `counter` = false
Counts down the delay between bullets for double barrel alternate fire.
- double `firerate` = 0.7
The turret's firing rate in seconds.
- bool `seesPlayer` = false

Indicates whether the turret can see the player.

- PackedScene `bullet` = GD.Load<PackedScene>("res://projectiles/projectile.tscn")

The packed scene that displays the bullet.

- bool `canFire` = true

Indicates whether the turret can fire.

8.2.1 Detailed Description

Represents a boss turret.

Definition at line 7 of file [Bossturret.cs](#).

8.2.2 Member Function Documentation

PhysicsProcess()

```
override void Bossturret._PhysicsProcess (
    double delta) [inline]
```

Godot update function. Gets called every frame to update the turret's logic.

Definition at line 43 of file [Bossturret.cs](#).

```
00043     {
00044         if (!GetNode<BossHealth>("Health").health._alive) {
00045             hasWeapon = false;
00046             if (!counter)
00047             {
00048                 GetParent().GetNode<Player>("Player").IncreaseBossCount();
00049                 counter = true;
00050             }
00051             GetNode<Sprite2D>("Turret").Visible = false;
00052         }
00053
00054         seesPlayer = CheckLineOfSight();
00055         if (hasWeapon)
00056         {
00057             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00058             if (seesPlayer && canFire)
00059             {
00060                 Fire1();
00061                 Fire2();
00062             }
00063         }
00064     }
```

CheckLineOfSight()

```
bool Bossturret.CheckLineOfSight () [inline]
```

Checks if the turret can see the player.

Returns

True if the turret can see the player, otherwise false.

Definition at line 100 of file [Bossturret.cs](#).

```
00101     {
00102         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00103         double distance = Mathf.Sqrt((playerPos.X - Position.X) * (playerPos.X - Position.X) +
00104                                     (playerPos.Y - Position.Y) * (playerPos.Y - Position.Y));
00105         if (distance < 300)
00106         {
00107             return true;
00108         }
00109         return false;
00110     }
```

Fire1()

```
async void Bossturret.Fire1 () [inline]
```

Fires the first bullet of the turret's weapon.

Definition at line 69 of file [Bossturret.cs](#).

```
00070     {
00071         GetNode<AudioStreamPlayer>("BulletSound").Play();
00072         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00073         GetParent().AddChild(bulletInstance);
00074         bulletInstance.Position = GetNode<Node2D>("BulletPoint1").GlobalPosition;
00075         bulletInstance.RotationDegrees = RotationDegrees;
00076         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00077         canFire = false;
00078         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00079         canFire = true;
00080     }
```

Fire2()

```
async void Bossturret.Fire2 () [inline]
```

Fires the second bullet of the turret's weapon.

Definition at line 85 of file [Bossturret.cs](#).

```
00086     {
00087         await ToSignal(GetTree().CreateTimer(0.35), "timeout");
00088         GetNode<AudioStreamPlayer>("BulletSound").Play();
00089         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00090         GetParent().AddChild(bulletInstance);
00091         bulletInstance.Position = GetNode<Node2D>("BulletPoint2").GlobalPosition;
00092         bulletInstance.RotationDegrees = RotationDegrees;
00093         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00094     }
```

8.2.3 Member Data Documentation**bullet**

```
PackedScene Bossturret.bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn")
[private]
```

The packed scene that displays the bullet.

Definition at line 32 of file [Bossturret.cs](#).

canFire

```
bool Bossturret.canFire = true [private]
```

Indicates whether the turret can fire.

Definition at line 37 of file [Bossturret.cs](#).

counter

```
bool Bossturret.counter = false [private]
```

Counts down the delay between bullets for double barrel alternate fire.

Definition at line 17 of file [Bossturret.cs](#).

firerate

```
double Bossturret.firerate = 0.7 [private]
```

The turret's firing rate in seconds.

Definition at line 22 of file [Bossturret.cs](#).

hasWeapon

```
bool Bossturret.hasWeapon = true [private]
```

Indicates whether the turret has a weapon.

Definition at line 12 of file [Bossturret.cs](#).

seesPlayer

```
bool Bossturret.seesPlayer = false [private]
```

Indicates whether the turret can see the player.

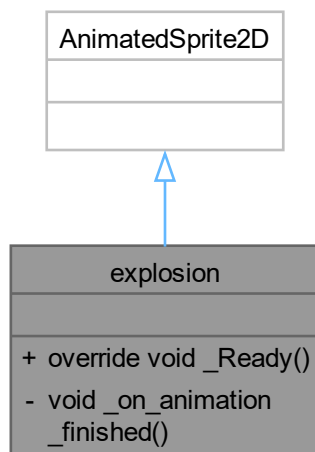
Definition at line 27 of file [Bossturret.cs](#).

The documentation for this class was generated from the following file:

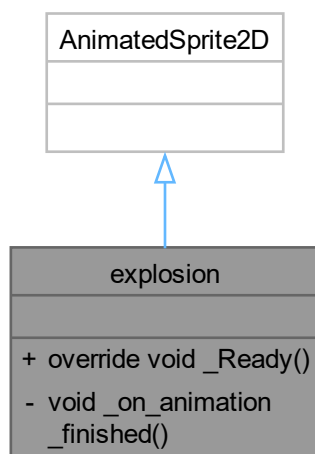
- [characters/Bossturret.cs](#)

8.3 explosion Class Reference

Inheritance diagram for explosion:



Collaboration diagram for explosion:



Public Member Functions

- override void `_Ready` ()

Private Member Functions

- void [_on_animation_finished](#) ()

8.3.1 Detailed Description

Definition at line 4 of file [explosion.cs](#).

8.3.2 Member Function Documentation

[_on_animation_finished\(\)](#)

```
void explosion._on_animation_finished () [inline], [private]
```

Definition at line 11 of file [explosion.cs](#).

```
00011                                     {
00012         QueueFree ();
00013     }
```

[_Ready\(\)](#)

```
override void explosion._Ready () [inline]
```

Definition at line 6 of file [explosion.cs](#).

```
00006                                     {
00007         Position = new Vector2(0,0);
00008         //Position = GetParent().GlobalPosition;
00009     }
```

The documentation for this class was generated from the following file:

- [projectiles/explosion.cs](#)

8.4 Health Class Reference

This class models the HP of an entity, health, shield, resistances, and vulnerabilities.

Collaboration diagram for Health:

Health
+ const int _damageBias
+ int _hp
+ int _hpMax
+ int _shield
+ int _shieldMax
+ int _resistancePhysical
+ int _resistanceElectric
+ int _vulnPhysical
+ int _vulnElectric
+ bool _alive
+ Health(int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
+ void ApplyTrueDamage(int damage)
+ void ApplyDamage(int damagePhysical, int damageElectric)
+ void HealHP(int amount)
+ void HealShield(int amount)
+ void HealBoth(int healAmount)
+ void DestroyShield()
+ void RestoreHealth()
+ void RestoreShield()
+ void damageShield(int damage)
and 14 more...

Public Member Functions

- [Health](#) (int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
- void [ApplyTrueDamage](#) (int damage)
- void [ApplyDamage](#) (int damagePhysical, int damageElectric)
- void [HealHP](#) (int amount)
- void [HealShield](#) (int amount)

- void `HealBoth` (int healAmount)
- void `DestroyShield` ()
- void `RestoreHealth` ()
- void `RestoreShield` ()
- void `damageShield` (int damage)
- void `damageHP` (int damage)
- void `Revive` ()
- `Health` (int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
Initializes a new instance of the `Health` class.
- void `ApplyTrueDamage` (int damage)
Applies true damage to shield and HP, ignoring resistances.
- void `ApplyDamage` (int damagePhysical, int damageElectric)
Applies physical and electric damage, taking into account resistances and vulnerabilities.
- void `HealHP` (int amount)
Heals a specified amount of HP.
- void `HealShield` (int amount)
Heals a specified amount of shield.
- void `HealBoth` (int healAmount)
Heals both HP and shield, prioritizing HP.
- void `DestroyShield` ()
Completely destroys the shield.
- void `RestoreHealth` ()
Restores HP to maximum.
- void `RestoreShield` ()
Restores shield to maximum.
- void `damageShield` (int damage)
Damages the shield by a specified amount.
- void `damageHP` (int damage)
Damages HP by a specified amount.
- void `Revive` ()
Revives the entity with 1 HP and no shield.

Static Public Attributes

- const int `_damageBias` = 12
Damage bias applied to all damage calculations.

Properties

- int `_hp` [get, private set]
Current hit points.
- int `_hpMax` [get, private set]
Maximum hit points.
- int `_shield` [get, private set]
Current shield points.
- int `_shieldMax` [get, private set]
Maximum shield points.
- int `_resistancePhysical` [get, private set]
Resistance against physical damage.

- `int _resistanceElectric` [get, private set]
Resistance against electric damage.
- `int _vulnPhysical` [get, private set]
Vulnerability to physical damage.
- `int _vulnElectric` [get, private set]
Vulnerability to electric damage.
- `bool _alive = true` [get, private set]
Indicates whether the entity is alive.

8.4.1 Detailed Description

This class models the HP of an entity, health, shield, resistances, and vulnerabilities.

Definition at line 3 of file [Health.cs](#).

8.4.2 Constructor & Destructor Documentation

Health() [1/2]

```
Health.Health (
    int hpMax,
    int shieldMax,
    int initialResistancePhysical,
    int initialResistanceElectric,
    int vulnPhysical,
    int vulnElectric) [inline]
```

Definition at line 16 of file [Health.cs](#).

```
00017 {
00018     _hpMax = hpMax;
00019     _shieldMax = shieldMax;
00020     _hp = _hpMax;
00021     _shield = _hpMax;
00022     _resistancePhysical = initialResistancePhysical;
00023     _resistanceElectric = initialResistanceElectric;
00024     _vulnElectric = vulnElectric;
00025     _vulnPhysical = vulnPhysical;
00026 }
```

Health() [2/2]

```
Health.Health (
    int hpMax,
    int shieldMax,
    int initialResistancePhysical,
    int initialResistanceElectric,
    int vulnPhysical,
    int vulnElectric) [inline]
```

Initializes a new instance of the [Health](#) class.

Parameters

<i>hpMax</i>	Maximum hit points.
<i>shieldMax</i>	Maximum shield points.

<i>initialResistancePhysical</i>	Initial physical resistance.
<i>initialResistanceElectric</i>	Initial electric resistance.
<i>vulnPhysical</i>	Physical vulnerability.
<i>vulnElectric</i>	Electric vulnerability.

Definition at line 67 of file [Health.cs](#).

```
00068     {
00069         _hpMax = hpMax;
00070         _shieldMax = shieldMax;
00071         _hp = _hpMax;
00072         _shield = _hpMax;
00073         _resistancePhysical = initialResistancePhysical;
00074         _resistanceElectric = initialResistanceElectric;
00075         _vulnElectric = vulnElectric;
00076         _vulnPhysical = vulnPhysical;
00077     }
```

8.4.3 Member Function Documentation

ApplyDamage() [1/2]

```
void Health.ApplyDamage (
    int damagePhysical,
    int damageElectric) [inline]
```

Definition at line 40 of file [Health.cs](#).

```
00041     {
00042         // line-by-line for easy debugging
00043         // doing stuff just twice doesn't warrant its own function
00044
00045         int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046         int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048         int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049         int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051         int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053         ApplyTrueDamage(damageTotal);
00054     }
```

ApplyDamage() [2/2]

```
void Health.ApplyDamage (
    int damagePhysical,
    int damageElectric) [inline]
```

Applies physical and electric damage, taking into account resistances and vulnerabilities.

Parameters

<i>damagePhysical</i>	Physical damage amount.
<i>damageElectric</i>	Electric damage amount.

Definition at line 101 of file [Health.cs](#).

```
00102     {
00103         // line-by-line for easy debugging
00104         // doing stuff just twice doesn't warrant its own function
00105
00106         int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00107         int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00108
00109         int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00110         int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00111
00112         int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00113
00114         ApplyTrueDamage(damageTotal);
00115     }
```

ApplyTrueDamage() [1/2]

```
void Health.ApplyTrueDamage (
    int damage) [inline]
```

Definition at line 28 of file [Health.cs](#).

```
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;
00033
00034         if (_hp <= 0)
00035         {
00036             _hp = 0;
00037             _alive = false;
00038         }
00039     }
```

ApplyTrueDamage() [2/2]

```
void Health.ApplyTrueDamage (
    int damage) [inline]
```

Applies true damage to shield and HP, ignoring resistances.

Parameters

<i>damage</i>	The amount of true damage to apply.
---------------	-------------------------------------

Definition at line 83 of file [Health.cs](#).

```
00084     {
00085         int damageAfterShield = Math.Max(damage - _shield, 0);
00086         _shield = Math.Max(_shield - damage, 0);
00087         _hp = _hp - damageAfterShield;
00088
00089         if (_hp <= 0)
00090         {
00091             _hp = 0;
00092             _alive = false;
00093         }
00094     }
```

damageHP() [1/2]

```
void Health.damageHP (
    int damage) [inline]
```

Definition at line 97 of file [Health.cs](#).

```
00098     {
00099         _hp = Math.Max(_hp - damage, 0);
00100         _alive = Convert.ToBoolean(_hp);
00101     }
```

damageHP() [2/2]

```
void Health.damageHP (
    int damage) [inline]
```

Damages HP by a specified amount.

Parameters

<i>damage</i>	Amount of damage to apply to HP.
---------------	----------------------------------

Definition at line 188 of file [Health.cs](#).

```
00189     {
00190         _hp = Math.Max(_hp - damage, 0);
00191         _alive = Convert.ToBoolean(_hp);
00192     }
```

damageShield() [1/2]

```
void Health.damageShield (
    int damage) [inline]
```

Definition at line 92 of file [Health.cs](#).

```
00093     {
00094         _shield = Math.Max(_shield - damage, 0);
00095     }
```

damageShield() [2/2]

```
void Health.damageShield (
    int damage) [inline]
```

Damages the shield by a specified amount.

Parameters

<i>damage</i>	Amount of damage to apply to the shield.
---------------	--

Definition at line 179 of file [Health.cs](#).

```
00180     {
00181         _shield = Math.Max(_shield - damage, 0);
00182     }
```

DestroyShield() [1/2]

```
void Health.DestroyShield () [inline]
```

Definition at line 78 of file [Health.cs](#).

```
00079     {
00080         _shield = 0;
00081     }
```

DestroyShield() [2/2]

```
void Health.DestroyShield () [inline]
```

Completely destroys the shield.

Definition at line 154 of file [Health.cs](#).

```
00155     {
00156         _shield = 0;
00157     }
```

HealBoth() [1/2]

```
void Health.HealBoth (
    int healAmount) [inline]
```

Definition at line 66 of file [Health.cs](#).

```
00067     {
00068         int healthMissing = _hpMax - _hp;
00069
00070         _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072         if (healAmount > healthMissing)
00073         {
00074             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075         }
00076     }
```

HealBoth() [2/2]

```
void Health.HealBoth (
    int healAmount) [inline]
```

Heals both HP and shield, prioritizing HP.

Parameters

<i>healAmount</i>	Total amount to heal.
-------------------	-----------------------

Definition at line 139 of file [Health.cs](#).

```
00140     {
00141         int healthMissing = _hpMax - _hp;
00142
00143         _hp = Math.Min(_hp + healAmount, _hpMax);
00144
00145         if (healAmount > healthMissing)
00146         {
00147             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00148         }
00149     }
```

HealHP() [1/2]

```
void Health.HealHP (
    int amount) [inline]
```

Definition at line 56 of file [Health.cs](#).

```
00057     {
00058         _hp = Math.Min(_hp + amount, _hpMax);
00059     }
```

HealHP() [2/2]

```
void Health.HealHP (
    int amount) [inline]
```

Heals a specified amount of HP.

Parameters

<i>amount</i>	Amount of HP to heal.
---------------	-----------------------

Definition at line 121 of file [Health.cs](#).

```
00122     {
00123         _hp = Math.Min(_hp + amount, _hpMax);
00124     }
```

HealShield() [1/2]

```
void Health.HealShield (
    int amount) [inline]
```

Definition at line 61 of file [Health.cs](#).

```
00062     {
00063         _shield = Math.Min(_shield + amount, _shieldMax);
00064     }
```

HealShield() [2/2]

```
void Health.HealShield (
    int amount) [inline]
```

Heals a specified amount of shield.

Parameters

<i>amount</i>	Amount of shield to heal.
---------------	---------------------------

Definition at line 130 of file [Health.cs](#).

```
00131     {
00132         _shield = Math.Min(_shield + amount, _shieldMax);
00133     }
```

RestoreHealth() [1/2]

```
void Health.RestoreHealth () [inline]
```

Definition at line 82 of file [Health.cs](#).

```
00083     {
00084         _hp = _hpMax;
00085     }
```

RestoreHealth() [2/2]

```
void Health.RestoreHealth () [inline]
```

Restores HP to maximum.

Definition at line 162 of file [Health.cs](#).

```
00163     {
00164         _hp = _hpMax;
00165     }
```

RestoreShield() [1/2]

```
void Health.RestoreShield () [inline]
```

Definition at line 87 of file [Health.cs](#).

```
00088     {  
00089         _shield = _shieldMax;  
00090     }
```

RestoreShield() [2/2]

```
void Health.RestoreShield () [inline]
```

Restores shield to maximum.

Definition at line 170 of file [Health.cs](#).

```
00171     {  
00172         _shield = _shieldMax;  
00173     }
```

Revive() [1/2]

```
void Health.Revive () [inline]
```

Definition at line 103 of file [Health.cs](#).

```
00104     {  
00105         _hp = 1;  
00106         _shield = 0;  
00107         _alive = true;  
00108     }
```

Revive() [2/2]

```
void Health.Revive () [inline]
```

Revives the entity with 1 HP and no shield.

Definition at line 197 of file [Health.cs](#).

```
00198     {  
00199         _hp = 1;  
00200         _shield = 0;  
00201         _alive = true;  
00202     }
```

8.4.4 Member Data Documentation

_damageBias

```
const int Health._damageBias = 12 [static]
```

Damage bias applied to all damage calculations.

Definition at line 14 of file [Health.cs](#).

8.4.5 Property Documentation

`_alive`

```
bool Health._alive = true [get], [private set]
```

Indicates whether the entity is alive.

Definition at line 13 of file [Health.cs](#).

```
00013 { get; private set; } = true;
```

`_hp`

```
int Health._hp [get], [private set]
```

Current hit points.

Definition at line 5 of file [Health.cs](#).

```
00005 { get; private set; }
```

`_hpMax`

```
int Health._hpMax [get], [private set]
```

Maximum hit points.

Definition at line 6 of file [Health.cs](#).

```
00006 { get; private set; }
```

`_resistanceElectric`

```
int Health._resistanceElectric [get], [private set]
```

Resistance against electric damage.

Definition at line 10 of file [Health.cs](#).

```
00010 { get; private set; }
```

`_resistancePhysical`

```
int Health._resistancePhysical [get], [private set]
```

Resistance against physical damage.

Definition at line 9 of file [Health.cs](#).

```
00009 { get; private set; }
```

_shield

```
int Health._shield [get], [private set]
```

Current shield points.

Definition at line 7 of file [Health.cs](#).

```
00007 { get; private set; }
```

_shieldMax

```
int Health._shieldMax [get], [private set]
```

Maximum shield points.

Definition at line 8 of file [Health.cs](#).

```
00008 { get; private set; }
```

_vulnElectric

```
int Health._vulnElectric [get], [private set]
```

Vulnerability to electric damage.

Definition at line 12 of file [Health.cs](#).

```
00012 { get; private set; }
```

_vulnPhysical

```
int Health._vulnPhysical [get], [private set]
```

Vulnerability to physical damage.

Definition at line 11 of file [Health.cs](#).

```
00011 { get; private set; }
```

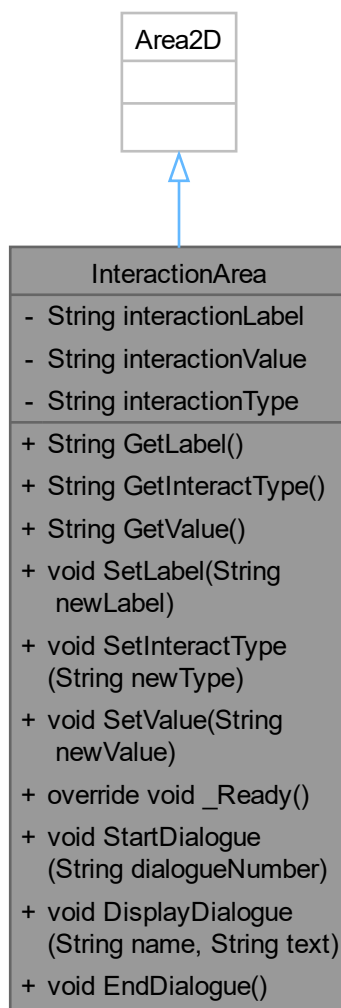
The documentation for this class was generated from the following files:

- game-systems/TestProject1/Health.cs
- systems/TestProject1/Health.cs

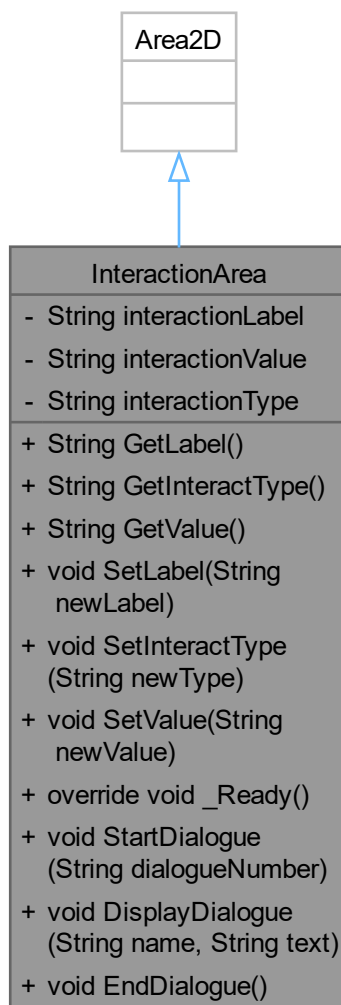
8.5 InteractionArea Class Reference

Represents an interaction area that triggers events when the player enters it.

Inheritance diagram for InteractionArea:



Collaboration diagram for InteractionArea:



Public Member Functions

- String [GetLabel](#) ()
Gets the label displayed for the interaction area.
- String [GetInteractType](#) ()
Gets the type of interaction.
- String [GetValue](#) ()
Gets the value associated with the interaction.
- void [SetLabel](#) (String newLabel)
Sets the label displayed for the interaction area.
- void [SetInteractType](#) (String newType)
Sets the type of interaction.
- void [SetValue](#) (String newValue)

- *Sets the value associated with the interaction.*
• override void `_Ready` ()
Initializes the interaction area.
- void `StartDialogue` (String dialogueNumber)
Starts a dialogue based on the given dialogue number.
- void `DisplayDialogue` (String name, String text)
Displays a dialogue with the given name and text.
- void `EndDialogue` ()
Ends the current dialogue and removes the dialogue manager.

Private Attributes

- String `interactionLabel` = "none"
The label displayed for the interaction area.
- String `interactionValue` = "none"
The value associated with the interaction.
- String `interactionType` = "none"
The type of interaction (e.g., "collect", "next_level", "dialogue").

8.5.1 Detailed Description

Represents an interaction area that triggers events when the player enters it.

Definition at line 7 of file [InteractionArea.cs](#).

8.5.2 Member Function Documentation

`_Ready()`

```
override void InteractionArea._Ready () [inline]
```

Initializes the interaction area.

Definition at line 84 of file [InteractionArea.cs](#).

```
00085     {
00086         GetNode<NinePatchRect>("DialogueManager").Visible = false;
00087     }
```

`DisplayDialogue()`

```
void InteractionArea.DisplayDialogue (
    String name,
    String text) [inline]
```

Displays a dialogue with the given name and text.

Parameters

<i>name</i>	The name to display.
<i>text</i>	The text to display.

Definition at line 124 of file [InteractionArea.cs](#).

```
00125     {
00126         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Name").Text = name;
00127         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Text").Text = text;
00128     }
```

EndDialogue()

```
void InteractionArea.EndDialogue () [inline]
```

Ends the current dialogue and removes the dialogue manager.

Definition at line 133 of file [InteractionArea.cs](#).

```
00134     {  
00135         GetNode<NinePatchRect>("DialogueManager").Visible = false;  
00136         QueueFree();  
00137     }
```

GetInteractType()

```
String InteractionArea.GetInteractType () [inline]
```

Gets the type of interaction.

Returns

The interaction type.

Definition at line 40 of file [InteractionArea.cs](#).

```
00041     {  
00042         return interactionType;  
00043     }
```

GetLabel()

```
String InteractionArea.GetLabel () [inline]
```

Gets the label displayed for the interaction area.

Returns

The interaction label.

Definition at line 31 of file [InteractionArea.cs](#).

```
00032     {  
00033         return interactionLabel;  
00034     }
```

GetValue()

```
String InteractionArea.GetValue () [inline]
```

Gets the value associated with the interaction.

Returns

The interaction value.

Definition at line 49 of file [InteractionArea.cs](#).

```
00050     {  
00051         return interactionValue;  
00052     }
```

SetInteractType()

```
void InteractionArea.SetInteractType (  
    String newType) [inline]
```

Sets the type of interaction.

Parameters

<i>newType</i>	The new interaction type.
----------------	---------------------------

Definition at line 67 of file [InteractionArea.cs](#).

```
00068     {  
00069         interactionType = newType;  
00070     }
```

SetLabel()

```
void InteractionArea.SetLabel (  
    String newLabel) [inline]
```

Sets the label displayed for the interaction area.

Parameters

<i>newLabel</i>	The new label.
-----------------	----------------

Definition at line 58 of file [InteractionArea.cs](#).

```
00059     {  
00060         interactionLabel = newLabel;  
00061     }
```

SetValue()

```
void InteractionArea.SetValue (  
    String newValue) [inline]
```

Sets the value associated with the interaction.

Parameters

<i>newValue</i>	The new interaction value.
-----------------	----------------------------

Definition at line 76 of file [InteractionArea.cs](#).

```
00077     {  
00078         interactionValue = newValue;  
00079     }
```

StartDialogue()

```
void InteractionArea.StartDialogue (  
    String dialogueNumber) [inline]
```

Starts a dialogue based on the given dialogue number.

Parameters

<i>dialogueNumber</i>	The dialogue number.
-----------------------	----------------------

Definition at line 93 of file [InteractionArea.cs](#).

```

00094     {
00095         GetNode<NinePatchRect>("DialogueManager").Visible = true;
00096
00097         switch (dialogueNumber)
00098         {
00099             case "0":
00100                 DisplayDialogue("Info!", "Welcome to Heidelberg Dynamics! You are L4lk4, and need to
escape this facility." +
00101                               " Use [W][A][S][D] to move to the next room. Press [E] to
continue.");
00102                 break;
00103             case "1":
00104                 DisplayDialogue("Info!", "You need to be careful. This facility is full of enemy
turrets that will" +
00105                               " shoot you on sight. Use [LMB] to shoot your weapon.");
00106                 break;
00107             case "2":
00108                 DisplayDialogue("Info!", "You need to collect a keycard to open the door to the next
level.");
00109                 break;
00110             case "3":
00111                 DisplayDialogue("Info!", "You escaped! You can now enjoy your freedom.");
00112                 break;
00113             default:
00114                 EndDialogue();
00115                 break;
00116         }
00117     }

```

8.5.3 Member Data Documentation

interactionLabel

```
String InteractionArea.interactionLabel = "none" [private]
```

The label displayed for the interaction area.

Definition at line 13 of file [InteractionArea.cs](#).

interactionType

```
String InteractionArea.interactionType = "none" [private]
```

The type of interaction (e.g., "collect", "next_level", "dialogue").

Definition at line 25 of file [InteractionArea.cs](#).

interactionValue

```
String InteractionArea.interactionValue = "none" [private]
```

The value associated with the interaction.

Definition at line 19 of file [InteractionArea.cs](#).

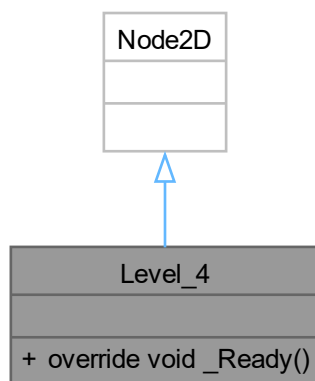
The documentation for this class was generated from the following file:

- [InteractionArea.cs](#)

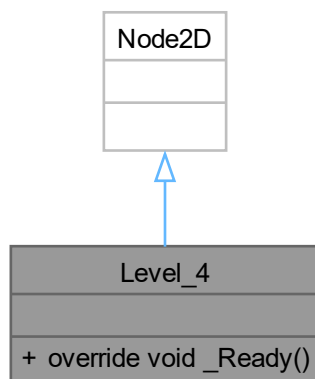
8.6 Level_4 Class Reference

Represents the logic for level 4 of the game.

Inheritance diagram for Level_4:



Collaboration diagram for Level_4:



Public Member Functions

- `override void \_Ready ()`
Initializes the level.

8.6.1 Detailed Description

Represents the logic for level 4 of the game.

Definition at line 7 of file [level_4.cs](#).

8.6.2 Member Function Documentation

_Ready()

```
override void Level_4._Ready () [inline]
```

Initializes the level.

Definition at line 12 of file [level_4.cs](#).

```
00013     {
00014         // Reset the player's extra card state to false
00015         GetNode<Player>("Player").SetExtraCard(false);
00016
00017         // Set the interaction value of the keycard2 to "keycard2"
00018         GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetValue("keycard2");
00019     }
```

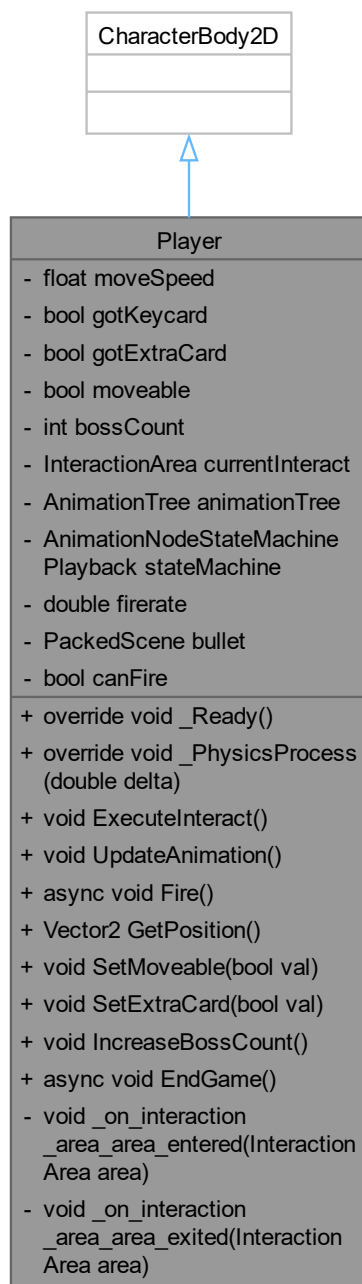
The documentation for this class was generated from the following file:

- levels/level_4.cs

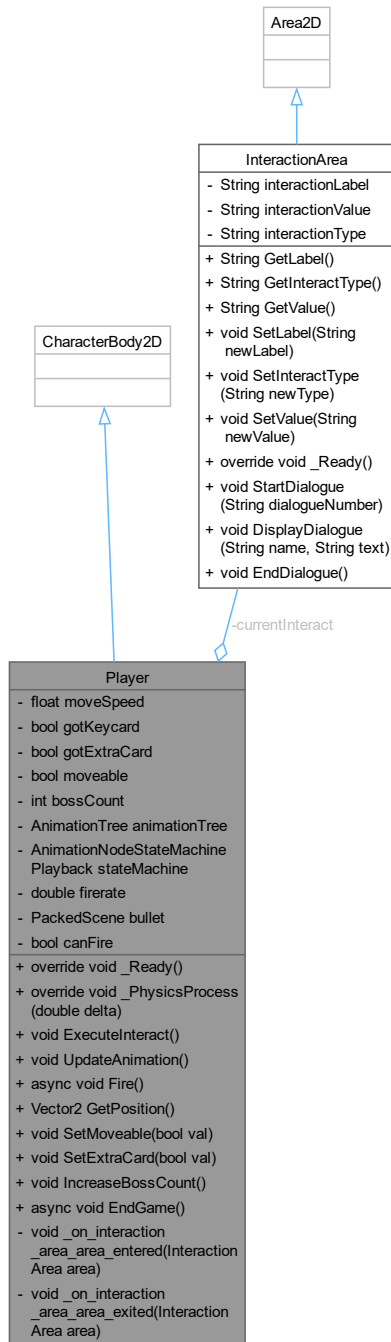
8.7 Player Class Reference

This class represents the player character in the game.

Inheritance diagram for Player:



Collaboration diagram for Player:



Public Member Functions

- `override void \_Ready ()`
Initializes the player character.
- `override void \_PhysicsProcess (double delta)`
Handles the player's physics processing and reacts to player inputs.
- `void ExecuteInteract ()`

- Determines and executes the current interaction, if any.*

 - void [UpdateAnimation](#) ()

Updates the player's animation based on their velocity.
- async void [Fire](#) ()

Fires the player's weapon by creating a new projectile instance.
- Vector2 [GetPosition](#) ()

Gets the player's current position.
- void [SetMoveable](#) (bool val)

Sets the player's moveability.
- void [SetExtraCard](#) (bool val)

Sets the player's extra card state.
- void [IncreaseBossCount](#) ()

Increases the boss count.
- async void [EndGame](#) ()

Ends the game after a 10 second delay.

Private Member Functions

- void [_on_interaction_area_area_entered](#) ([InteractionArea](#) area)

Handles when the player enters an [InteractionArea](#) and displays the interaction information.
- void [_on_interaction_area_area_exited](#) ([InteractionArea](#) area)

Handles when the player exits an [InteractionArea](#) and resets the according values.

Private Attributes

- float [moveSpeed](#) = 175

The player's movement speed.
- bool [gotKeycard](#) = false

Indicates if the player has collected the keycard.
- bool [gotExtraCard](#) = true

Indicates if the player has collected the extra keycard.
- bool [moveable](#) = true

Determines if the player can move.
- int [bossCount](#) = 0

Keeps track of the number of bosses defeated.
- [InteractionArea](#) [currentInteract](#)

Reference to the currently interacted [InteractionArea](#).
- AnimationTree [animationTree](#)

Reference to the player's AnimationTree.
- AnimationNodeStateMachinePlayback [stateMachine](#)

Reference to the AnimationNodeStateMachinePlayback node.
- double [firerate](#) = 0.5

The fire rate of the player's weapon in seconds.
- PackedScene [bullet](#) = GD.Load<PackedScene>("res://projectiles/projectile.tscn")

PackedScene reference to the projectile scene.
- bool [canFire](#) = true

Indicates if the player can fire.

8.7.1 Detailed Description

This class represents the player character in the game.

Definition at line 7 of file [Player.cs](#).

8.7.2 Member Function Documentation

_on_interaction_area_area_entered()

```
void Player._on_interaction_area_area_entered (
    InteractionArea area) [inline], [private]
```

Handles when the player enters an [InteractionArea](#) and displays the interaction information.

Parameters

<i>area</i>	The InteractionArea the player entered.
-------------	---

Definition at line 235 of file [Player.cs](#).

```
00236     {
00237         if (area.GetInteractType() != "heal")
00238         {
00239             this.currentInteract = area;
00240             this.currentInteract.GetNode<Label>("Label").Text = currentInteract.GetLabel();
00241             if (area.GetInteractType() == "dialogue")
00242             {
00243                 area.StartDialogue(area.GetValue());
00244                 moveable = false;
00245             }
00246         }
00247         else
00248         {
00249             GetNode<PlayerHealth>("Health").health.HealBoth(500);
00250             area.GetParent().QueueFree();
00251         }
00252     }
```

_on_interaction_area_area_exited()

```
void Player._on_interaction_area_area_exited (
    InteractionArea area) [inline], [private]
```

Handles when the player exits an [InteractionArea](#) and resets the according values.

Parameters

<i>area</i>	The InteractionArea the player exited.
-------------	--

Definition at line 258 of file [Player.cs](#).

```
00259     {
00260         area.GetNode<Label>("Label").Text = "";
00261         this.currentInteract = null;
00262     }
```

_PhysicsProcess()

```
override void Player._PhysicsProcess (
    double delta) [inline]
```

Handles the player's physics processing and reacts to player inputs.

Parameters

<i>delta</i>	The elapsed time since the last frame.
--------------	--

Definition at line 84 of file [Player.cs](#).

```

00085     {
00086         // Update health bars based on PlayerHealth node
00087         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Health").Value =
GetNode<PlayerHealth>("Health").health._hp;
00088         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Shield").Value =
GetNode<PlayerHealth>("Health").health._shield;
00089
00090         // Check if player is dead and update visuals accordingly
00091         if (!GetNode<PlayerHealth>("Health").health._alive)
00092         {
00093             moveable = false;
00094             GetParent().GetNode<CanvasLayer>("HUD").GetNode<NinePatchRect>("Defeat").Visible = true;
00095         }
00096
00097         // Get input directions
00098         Vector2 inputDirection = new Vector2(
00099             Input.GetActionRawStrength("right") - Input.GetActionRawStrength("left"),
00100             Input.GetActionRawStrength("down") - Input.GetActionRawStrength("up")
00101         );
00102
00103         if (moveable)
00104         {
00105             // Look at the mouse position
00106             LookAt(GetGlobalMousePosition());
00107
00108             // Fire bullet if pressed and able to fire
00109             if (Input.IsActionPressed("fire") && canFire)
00110             {
00111                 Fire();
00112             }
00113
00114             // Update velocity and move character
00115             this.Velocity = inputDirection * moveSpeed;
00116             MoveAndSlide();
00117
00118             UpdateAnimation();
00119         }
00120
00121         // Execute interaction if needed
00122         if (Input.IsActionPressed("interact"))
00123         {
00124             if (GetNode<PlayerHealth>("Health").health._alive)
00125             {
00126                 ExecuteInteract();
00127             }
00128             else
00129             {
00130                 GetTree().ChangeSceneToFile("res://levels/level_1.tscn");
00131             }
00132         }
00133
00134         // Handle other input actions
00135         if (Input.IsActionPressed("exit"))
00136         {
00137             GetTree().Quit();
00138         }
00139
00140         if (Input.IsActionPressed("cheat"))
00141         {
00142             GetTree().ChangeSceneToFile("res://levels/level_6.tscn");
00143         }
00144
00145         if (Input.IsActionPressed("speed"))
00146         {
00147             moveSpeed = 500;
00148         }
00149
00150         // Check for endgame condition
00151         if (bossCount == 2)
00152         {
00153             EndGame();
00154         }
00155     }

```

_Ready()

```
override void Player._Ready () [inline]
```

Initializes the player character.

Definition at line 71 of file [Player.cs](#).

```
00072     {
00073         moveable = true;
00074         this.gotKeycard = false;
00075         animationTree = GetNode<AnimationTree>("AnimationTree");
00076         stateMachine = (AnimationNodeStateMachinePlayback)animationTree.Get("parameters/playback");
00077     }
00078 }
```

EndGame()

```
async void Player.EndGame () [inline]
```

Ends the game after a 10 second delay.

Definition at line 321 of file [Player.cs](#).

```
00322     {
00323         await ToSignal(GetTree().CreateTimer(10), "timeout");
00324         GetTree().ChangeSceneToFile("res://levels/level_7.tscn");
00325     }
```

ExecuteInteract()

```
void Player.ExecuteInteract () [inline]
```

Determines and executes the current interaction, if any.

Definition at line 160 of file [Player.cs](#).

```
00161     {
00162         if (currentInteract != null)
00163         {
00164             // Check which type of interaction is executed
00165             switch (currentInteract.GetInteractType())
00166             {
00167                 case "collect":
00168                     // Check if a keycard or a weapon is picked up
00169                     if (currentInteract.GetValue() == "keycard")
00170                     {
00171                         this.gotKeycard = true;
00172                         GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D").Visible
00173                         = true;
00174                         // Update interaction labels and visibility of keycard
00175                         GetParent().GetNode<StaticBody2D>("keycard").Visible = false;
00176                         GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00177                         GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00178                         // Change label of door
00179                         if (gotExtraCard)
00180                         {
00181                             GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
00182                             next level");
00183                         }
00184                     }
00185                     else if (currentInteract.GetValue() == "keycard2")
00186                     {
00187                         this.gotExtraCard = true;
00188                         GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D2").Visible
00189                         = true;
00190                         // Update interaction labels and visibility of keycard
00191                         GetParent().GetNode<StaticBody2D>("keycard2").Visible = false;
00192                         GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00193                         GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00194                         // Change label of door
00195                         if (gotKeycard)
00196                         {
```

```

00197         {
00198             GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
next_level");
00199         }
00200     }
00201     break;
00202     case "next_level":
00203         if (gotKeycard && gotExtraCard)
00204         {
00205             GetTree().ChangeSceneToFile("res://levels/" +
GetParent().GetNode<InteractionArea>("NextLevel").GetValue());
00206         }
00207         break;
00208     case "dialogue":
00209         moveable = true;
00210         currentInteract.EndDialogue();
00211         break;
00212     }
00213 }
00214 }

```

Fire()

```
async void Player.Fire () [inline]
```

Fires the player's weapon by creating a new projectile instance.

Definition at line 267 of file [Player.cs](#).

```

00268     {
00269         GetNode<AudioStreamPlayer>("BulletSound").Play();
00270
00271         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00272         GetParent().AddChild(bulletInstance);
00273
00274         bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00275         bulletInstance.RotationDegrees = RotationDegrees;
00276         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00277
00278         canFire = false;
00279         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00280         canFire = true;
00281     }

```

GetPosition()

```
Vector2 Player.GetPosition () [inline]
```

Gets the player's current position.

Returns

The player's position.

Definition at line 287 of file [Player.cs](#).

```

00288     {
00289         return Position;
00290     }

```

IncreaseBossCount()

```
void Player.IncreaseBossCount () [inline]
```

Increases the boss count.

Definition at line 313 of file [Player.cs](#).

```

00314     {
00315         bossCount++;
00316     }

```

SetExtraCard()

```
void Player.SetExtraCard (
    bool val) [inline]
```

Sets the player's extra card state.

Parameters

<i>val</i>	The new extra card state.
------------	---------------------------

Definition at line 305 of file [Player.cs](#).

```
00306     {
00307         gotExtraCard = val;
00308     }
```

SetMoveable()

```
void Player.SetMoveable (
    bool val) [inline]
```

Sets the player's moveability.

Parameters

<i>val</i>	The new moveability state.
------------	----------------------------

Definition at line 296 of file [Player.cs](#).

```
00297     {
00298         moveable = val;
00299     }
```

UpdateAnimation()

```
void Player.UpdateAnimation () [inline]
```

Updates the player's animation based on their velocity.

Definition at line 219 of file [Player.cs](#).

```
00220     {
00221         if (this.Velocity != Vector2.Zero)
00222         {
00223             stateMachine.Travel("walk");
00224         }
00225         else
00226         {
00227             stateMachine.Travel("idle");
00228         }
00229     }
```

8.7.3 Member Data Documentation

animationTree

```
AnimationTree Player.animationTree [private]
```

Reference to the player's AnimationTree.

Definition at line 44 of file [Player.cs](#).

bossCount

```
int Player.bossCount = 0 [private]
```

Keeps track of the number of bosses defeated.

Definition at line 34 of file [Player.cs](#).

bullet

```
PackedScene Player.bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn") [private]
```

PackedScene reference to the projectile scene.

Definition at line 59 of file [Player.cs](#).

canFire

```
bool Player.canFire = true [private]
```

Indicates if the player can fire.

Definition at line 64 of file [Player.cs](#).

currentInteract

```
InteractionArea Player.currentInteract [private]
```

Reference to the currently interacted [InteractionArea](#).

Definition at line 39 of file [Player.cs](#).

fireRate

```
double Player.fireRate = 0.5 [private]
```

The fire rate of the player's weapon in seconds.

Definition at line 54 of file [Player.cs](#).

gotExtraCard

```
bool Player.gotExtraCard = true [private]
```

Indicates if the player has collected the extra keycard.

Definition at line 24 of file [Player.cs](#).

gotKeycard

```
bool Player.gotKeycard = false [private]
```

Indicates if the player has collected the keycard.

Definition at line 19 of file [Player.cs](#).

moveable

```
bool Player.moveable = true [private]
```

Determines if the player can move.

Definition at line 29 of file [Player.cs](#).

moveSpeed

```
float Player.moveSpeed = 175 [private]
```

The player's movement speed.

Definition at line 14 of file [Player.cs](#).

stateMachine

```
AnimationNodeStateMachinePlayback Player.stateMachine [private]
```

Reference to the AnimationNodeStateMachinePlayback node.

Definition at line 49 of file [Player.cs](#).

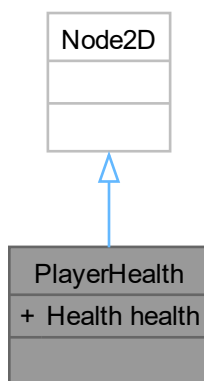
The documentation for this class was generated from the following file:

- [characters/Player.cs](#)

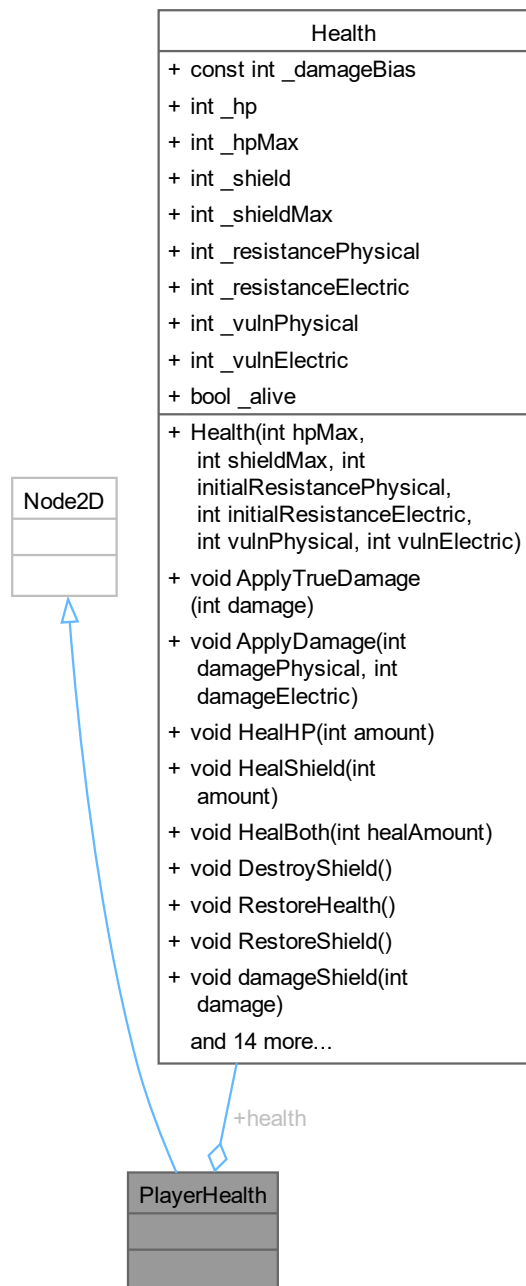
8.8 PlayerHealth Class Reference

Represents the health of the player character.

Inheritance diagram for PlayerHealth:



Collaboration diagram for PlayerHealth:



Public Attributes

- `Health health = new Health(1000, 1000, 100, 100, 100, 100)`
The player's health information.

8.8.1 Detailed Description

Represents the health of the player character.

Definition at line 7 of file [PlayerHealth.cs](#).

8.8.2 Member Data Documentation

health

```
Health PlayerHealth.health = new Health(1000, 1000, 100, 100, 100, 100)
```

The player's health information.

Definition at line 12 of file [PlayerHealth.cs](#).

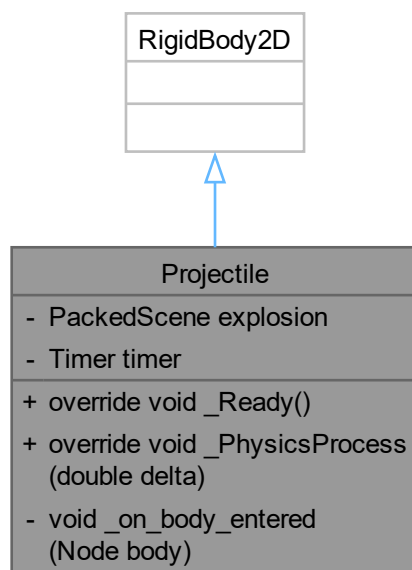
The documentation for this class was generated from the following file:

- characters/PlayerHealth.cs

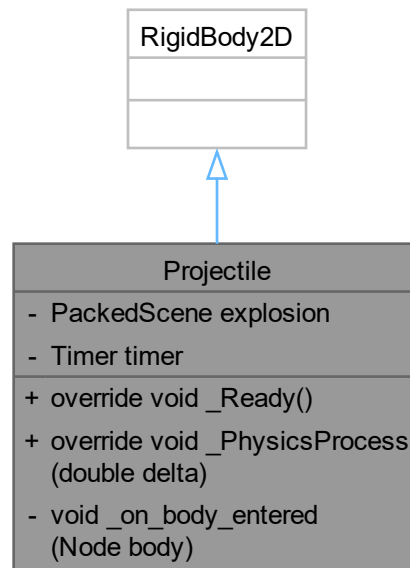
8.9 Projectile Class Reference

Represents a projectile entity in the game.

Inheritance diagram for Projectile:



Collaboration diagram for Projectile:



Public Member Functions

- override void `_Ready()`
Initializes the projectile.
- override void `_PhysicsProcess(double delta)`
Updates the projectile's state.

Private Member Functions

- void `_on_body_entered(Node body)`
Handles collisions with other game objects.

Private Attributes

- PackedScene `explosion` = GD.Load<PackedScene>("res://projectiles/explosion.tscn")
PackedScene reference to the explosion effect.
- Timer `timer`
Timer to manage the projectile's lifespan.

8.9.1 Detailed Description

Represents a projectile entity in the game.

Definition at line 7 of file [projectile.cs](#).

8.9.2 Member Function Documentation

_on_body_entered()

```
void Projectile._on_body_entered (
    Node body) [inline], [private]
```

Handles collisions with other game objects.

Parameters

<i>body</i>	The body that the projectile collided with.
-------------	---

Definition at line 36 of file [projectile.cs](#).

```
00037     {
00038         // Check if the collision is with a valid target
00039         if (!body.IsInGroup("entity"))
00040         {
00041             // If not, destroy the projectile
00042             QueueFree();
00043         }
00044         else if (body.IsInGroup("player"))
00045         {
00046             // If it's the player, damage them and play a hit sound
00047             QueueFree();
00048             body.GetNode<PlayerHealth>("Health").health.ApplyDamage(50, 50);
00049             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00050         }
00051         else if (body.IsInGroup("boss"))
00052         {
00053             // If it's a boss, damage them and play a hit sound
00054             QueueFree();
00055             body.GetNode<BossHealth>("Health").health.ApplyDamage(50, 50);
00056             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00057         }
00058         else
00059         {
00060             // If it's a turret, damage it and play a hit sound
00061             QueueFree();
00062             body.GetNode<TurretHealth>("Health").health.ApplyDamage(50, 50);
00063             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00064         }
00065     }
```

_PhysicsProcess()

```
override void Projectile._PhysicsProcess (
    double delta) [inline]
```

Updates the projectile's state.

Parameters

<i>delta</i>	The elapsed time since the last frame.
--------------	--

Definition at line 71 of file [projectile.cs](#).

```
00072     {
00073         // Destroy the projectile if the timer has expired
00074         if (timer.IsStopped())
00075         {
00076             QueueFree();
00077         }
00078     }
```

`_Ready()`

```
override void Projectile._Ready () [inline]
```

Initializes the projectile.

Definition at line 22 of file [projectile.cs](#).

```
00023     {
00024         // Create a timer to destroy the projectile after a set duration.
00025         timer = new Timer();
00026         this.AddChild(timer);
00027         timer.WaitTime = 0.34; // Adjust the lifespan as needed
00028         timer.OneShot = true;
00029         timer.Start();
00030     }
```

8.9.3 Member Data Documentation

explosion

```
PackedScene Projectile.explosion = GD.Load<PackedScene>("res://projectiles/explosion.tscn")
[private]
```

PackedScene reference to the explosion effect.

Definition at line 12 of file [projectile.cs](#).

timer

```
Timer Projectile.timer [private]
```

Timer to manage the projectile's lifespan.

Definition at line 17 of file [projectile.cs](#).

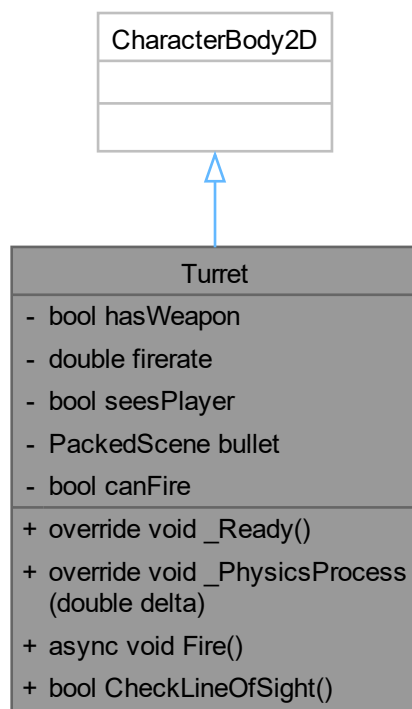
The documentation for this class was generated from the following file:

- [projectiles/projectile.cs](#)

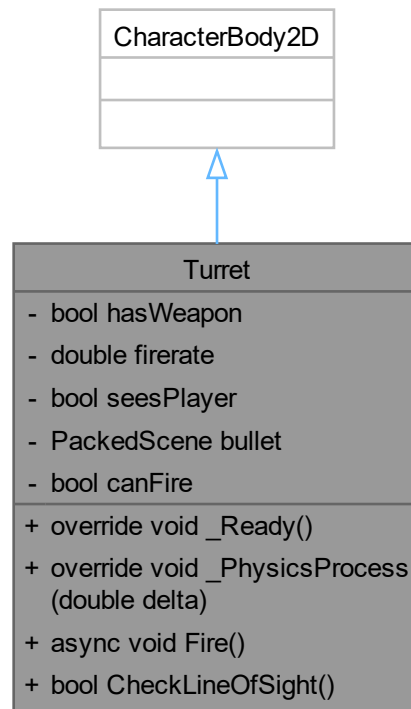
8.10 Turret Class Reference

This class represents a turret in a 2D game.

Inheritance diagram for Turret:



Collaboration diagram for Turret:



Public Member Functions

- override void `_Ready` ()
Initializes the turret.
- override void `_PhysicsProcess` (double delta)
Godot update function. Gets called every frame to update the turret's logic.
- async void `Fire` ()
Fires the turret's weapon.
- bool `CheckLineOfSight` ()
Checks if the turret can see the player.

Private Attributes

- bool `hasWeapon` = true
Indicates whether the turret has a weapon.
- double `firerate` = 0.65
The turret's firing rate in seconds.
- bool `seesPlayer` = false
Indicates whether the turret can see the player.
- PackedScene `bullet` = GD.Load<PackedScene>("res://projectiles/projectile.tscn")
The packed scene for the bullet.
- bool `canFire` = true
Indicates whether the turret can fire.

8.10.1 Detailed Description

This class represents a turret in a 2D game.

Definition at line 7 of file [turret.cs](#).

8.10.2 Member Function Documentation

PhysicsProcess()

```
override void Turret._PhysicsProcess (  
    double delta) [inline]
```

Godot update function. Gets called every frame to update the turret's logic.

Parameters

<i>delta</i>	The time delta in seconds.
--------------	----------------------------

Definition at line 45 of file [turret.cs](#).

```
00046     {  
00047         if (!GetNode<TurretHealth>("Health").health._alive)  
00048         {  
00049             hasWeapon = false;  
00050             GetNode<Sprite2D>("Turret").Visible = false;  
00051         }  
00052  
00053         seesPlayer = CheckLineOfSight();  
00054  
00055         if (hasWeapon)  
00056         {  
00057             LookAt(GetParent().GetNode<Player>("Player").GetPosition());  
00058  
00059             if (seesPlayer && canFire)  
00060             {  
00061                 Fire();  
00062             }  
00063         }  
00064     }
```

Ready()

```
override void Turret._Ready () [inline]
```

Initializes the turret.

Definition at line 37 of file [turret.cs](#).

```
00038     {  
00039     }
```

CheckLineOfSight()

```
bool Turret.CheckLineOfSight () [inline]
```

Checks if the turret can see the player.

Returns

True if the turret can see the player, otherwise false.

Definition at line 88 of file [turret.cs](#).

```
00089     {
00090         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00091         double distance = Mathf.Sqrt((playerPos.X - Position.X) * (playerPos.X - Position.X) +
00092                                     (playerPos.Y - Position.Y) * (playerPos.Y - Position.Y));
00093
00094         if (distance < 300)
00095         {
00096             return true;
00097         }
00098
00099         return false;
00100     }
```

Fire()

```
async void Turret.Fire () [inline]
```

Fires the turret's weapon.

Definition at line 69 of file [turret.cs](#).

```
00070     {
00071         GetNode<AudioStreamPlayer>("BulletSound").Play();
00072
00073         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00074         GetParent().AddChild(bulletInstance);
00075         bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00076         bulletInstance.RotationDegrees = RotationDegrees;
00077         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00078
00079         canFire = false;
00080         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00081         canFire = true;
00082     }
```

8.10.3 Member Data Documentation

bullet

```
PackedScene Turret.bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn") [private]
```

The packed scene for the bullet.

Definition at line 27 of file [turret.cs](#).

canFire

```
bool Turret.canFire = true [private]
```

Indicates whether the turret can fire.

Definition at line 32 of file [turret.cs](#).

fireRate

```
double Turret.fireRate = 0.65 [private]
```

The turret's firing rate in seconds.

Definition at line 17 of file [turret.cs](#).

hasWeapon

```
bool Turret.hasWeapon = true [private]
```

Indicates whether the turret has a weapon.

Definition at line 12 of file [turret.cs](#).

seesPlayer

```
bool Turret.seesPlayer = false [private]
```

Indicates whether the turret can see the player.

Definition at line 22 of file [turret.cs](#).

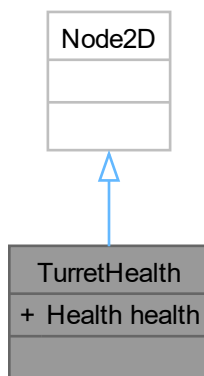
The documentation for this class was generated from the following file:

- [characters/turret.cs](#)

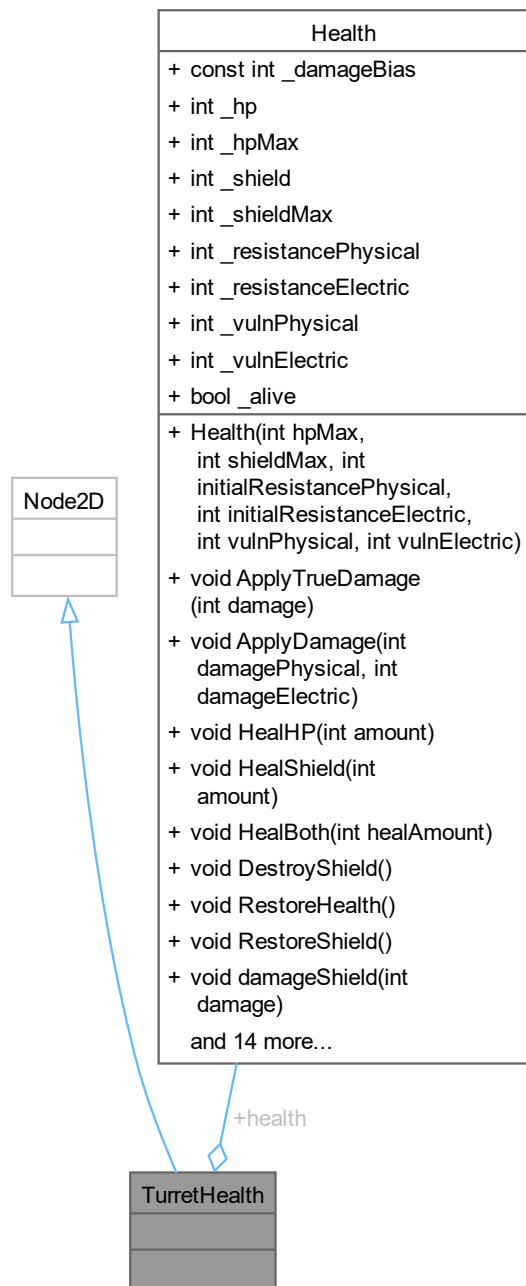
8.11 TurretHealth Class Reference

Represents the health of a turret in the game.

Inheritance diagram for TurretHealth:



Collaboration diagram for TurretHealth:



Public Attributes

- `Health health = new Health(100,100,100,100,100,100)`

The health object associated with the turret.

8.11.1 Detailed Description

Represents the health of a turret in the game.

Definition at line 7 of file [TurretHealth.cs](#).

8.11.2 Member Data Documentation

health

```
Health TurretHealth.health = new Health(100,100,100,100,100,100)
```

The health object associated with the turret.

Definition at line 12 of file [TurretHealth.cs](#).

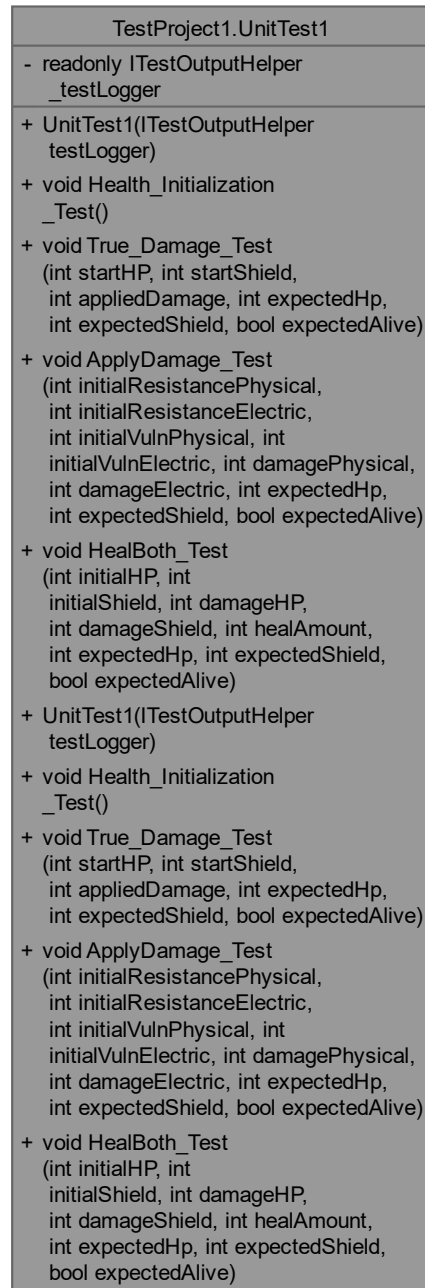
The documentation for this class was generated from the following file:

- [characters/TurretHealth.cs](#)

8.12 TestProject1.UnitTest1 Class Reference

Unit Tests for Health.cs.

Collaboration diagram for TestProject1.UnitTest1:



Public Member Functions

- [UnitTest1](#) (ITestOutputHelper testLogger)
- void [Health_Initialization_Test](#) ()
- void [True_Damage_Test](#) (int startHP, int startShield, int appliedDamage, int expectedHp, int expectedShield, bool expectedAlive)

- void [ApplyDamage_Test](#) (int initialResistancePhysical, int initialResistanceElectric, int initialVulnPhysical, int initialVulnElectric, int damagePhysical, int damageElectric, int expectedHp, int expectedShield, bool expectedAlive)
- void [HealBoth_Test](#) (int initialHP, int initialShield, int damageHP, int damageShield, int healAmount, int expectedHp, int expectedShield, bool expectedAlive)
- [UnitTest1](#) (ITestOutputHelper testLogger)
- void [Health_Initialization_Test](#) ()
Tests the constructor.
- void [True_Damage_Test](#) (int startHP, int startShield, int appliedDamage, int expectedHp, int expectedShield, bool expectedAlive)
Tests ApplyTrueDamage()
- void [ApplyDamage_Test](#) (int initialResistancePhysical, int initialResistanceElectric, int initialVulnPhysical, int initialVulnElectric, int damagePhysical, int damageElectric, int expectedHp, int expectedShield, bool expectedAlive)
Tests ApplyDamage()
- void [HealBoth_Test](#) (int initialHP, int initialShield, int damageHP, int damageShield, int healAmount, int expectedHp, int expectedShield, bool expectedAlive)
Tests HealBoth()

Private Attributes

- readonly ITestOutputHelper [_testLogger](#)

8.12.1 Detailed Description

Unit Tests for Health.cs.

Definition at line 8 of file [UnitTest1.cs](#).

8.12.2 Constructor & Destructor Documentation

UnitTest1() [1/2]

```
TestProject1.UnitTest1.UnitTest1 (
    ITestOutputHelper testLogger) [inline]
```

Definition at line 8 of file [UnitTest1.cs](#).

```
00009     {
00010         this._testLogger = testLogger;
00011     }
```

UnitTest1() [2/2]

```
TestProject1.UnitTest1.UnitTest1 (
    ITestOutputHelper testLogger) [inline]
```

Definition at line 11 of file [UnitTest1.cs](#).

```
00012     {
00013         this._testLogger = testLogger;
00014     }
```

8.12.3 Member Function Documentation

ApplyDamage_Test() [1/2]

```
void TestProject1.UnitTest1.ApplyDamage_Test (
    int initialResistancePhysical,
    int initialResistanceElectric,
    int initialVulnPhysical,
    int initialVulnElectric,
    int damagePhysical,
    int damageElectric,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 64 of file [UnitTest1.cs](#).

```
00069     {
00070         // Arrange
00071         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00072             initialVulnPhysical, initialVulnElectric);
00073
00074         // Act
00075         health.ApplyDamage(damagePhysical, damageElectric);
00076
00077         // Assert
00078         Assert.Equal(expectedHp, health._hp);
00079         Assert.Equal(expectedShield, health._shield);
00080         Assert.Equal(expectedAlive, health._alive);
00081     }
```

ApplyDamage_Test() [2/2]

```
void TestProject1.UnitTest1.ApplyDamage_Test (
    int initialResistancePhysical,
    int initialResistanceElectric,
    int initialVulnPhysical,
    int initialVulnElectric,
    int damagePhysical,
    int damageElectric,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Tests ApplyDamage()

Definition at line 79 of file [UnitTest1.cs](#).

```
00084     {
00085         // Arrange
00086         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00087             initialVulnPhysical, initialVulnElectric);
00088
00089         // Act
00090         health.ApplyDamage(damagePhysical, damageElectric);
00091
00092         // Assert
00093         Assert.Equal(expectedHp, health._hp);
00094         Assert.Equal(expectedShield, health._shield);
00095         Assert.Equal(expectedAlive, health._alive);
00096     }
```

HealBoth_Test() [1/2]

```
void TestProject1.UnitTest1.HealBoth_Test (
    int initialHP,
    int initialShield,
    int damageHP,
    int damageShield,
    int healAmount,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 92 of file [UnitTest1.cs](#).

```
00096     {
00097         // Arrange
00098         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00099
00100         // Act
00101         health.damageHP(damageHP);
00102         health.damageShield(damageShield);
00103
00104         health.HealBoth(healAmount);
00105
00106         // Assert
00107         Assert.Equal(expectedHp, health._hp);
00108         Assert.Equal(expectedShield, health._shield);
00109         Assert.Equal(expectedAlive, health._alive);
00110     }
```

HealBoth_Test() [2/2]

```
void TestProject1.UnitTest1.HealBoth_Test (
    int initialHP,
    int initialShield,
    int damageHP,
    int damageShield,
    int healAmount,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Tests HealBoth()

Definition at line 110 of file [UnitTest1.cs](#).

```
00114     {
00115         // Arrange
00116         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00117
00118         // Act
00119         health.damageHP(damageHP);
00120         health.damageShield(damageShield);
00121
00122         health.HealBoth(healAmount);
00123
00124         // Assert
00125         Assert.Equal(expectedHp, health._hp);
00126         Assert.Equal(expectedShield, health._shield);
00127         Assert.Equal(expectedAlive, health._alive);
00128     }
```

Health_Initialization_Test() [1/2]

```
void TestProject1.UnitTest1.Health_Initialization_Test () [inline]
```

Definition at line 14 of file [UnitTest1.cs](#).

```
00015     {
00016         int initialResistancePhysical = 100;
00017         int initialResistanceElectric = 300;
00018         int initialVulnPhysical = 0;
00019         int initialVulnElectric = 200;
00020
00021         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00022             initialVulnPhysical, initialVulnElectric);
00023
00024         Assert.Equal(1000, health._hpMax);
00025         Assert.Equal(1000, health._hp);
00026         Assert.Equal(1000, health._shield);
00027         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00028         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00029         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00030         Assert.Equal(initialVulnElectric, health._vulnElectric);
00031         Assert.True(health._alive);
00032     }
```

Health_Initialization_Test() [2/2]

```
void TestProject1.UnitTest1.Health_Initialization_Test () [inline]
```

Tests the constructor.

Definition at line 20 of file [UnitTest1.cs](#).

```
00021     {
00022         int initialResistancePhysical = 100;
00023         int initialResistanceElectric = 300;
00024         int initialVulnPhysical = 0;
00025         int initialVulnElectric = 200;
00026
00027         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00028             initialVulnPhysical, initialVulnElectric);
00029
00030         Assert.Equal(1000, health._hpMax);
00031         Assert.Equal(1000, health._hp);
00032         Assert.Equal(1000, health._shield);
00033         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00034         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00035         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00036         Assert.Equal(initialVulnElectric, health._vulnElectric);
00037         Assert.True(health._alive);
00038     }
```

True_Damage_Test() [1/2]

```
void TestProject1.UnitTest1.True_Damage_Test (
    int startHP,
    int startShield,
    int appliedDamage,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 41 of file [UnitTest1.cs](#).

```
00044     {
00045         // Arrange
00046         Health health = new(startHP, startShield, 1000, 1000, 0, 0);
00047
00048         // Act
00049         health.ApplyTrueDamage(appliedDamage);
00050
00051         // Assert
00052         Assert.Equal(expectedHp, health._hp);
00053         Assert.Equal(expectedShield, health._shield);
00054         Assert.Equal(expectedAlive, health._alive);
00055     }
```

True_Damage_Test() [2/2]

```
void TestProject1.UnitTest1.True_Damage_Test (
    int startHP,
    int startShield,
    int appliedDamage,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Tests ApplyTrueDamage()

Definition at line 53 of file [UnitTest1.cs](#).

```
00056     {
00057         // Arrange
00058         Health health = new(startHP, startShield, 1000, 1000, 0, 0);
00059
00060         // Act
00061         health.ApplyTrueDamage(appliedDamage);
00062
00063         // Assert
00064         Assert.Equal(expectedHp, health._hp);
00065         Assert.Equal(expectedShield, health._shield);
00066         Assert.Equal(expectedAlive, health._alive);
00067     }
```

8.12.4 Member Data Documentation**_testLogger**

```
readonly ITestOutputHelper TestProject1.UnitTest1._testLogger [private]
```

Definition at line 7 of file [UnitTest1.cs](#).

The documentation for this class was generated from the following files:

- game-systems/TestProject1/UnitTest1.cs
- systems/TestProject1/UnitTest1.cs

9 File Documentation**9.1 BossHealth.cs**

```
00001 using Godot;
00002 using System;
00003
00007 public partial class BossHealth : Node2D
00008 {
00012     public Health health = new Health(1000, 1000, 100, 100, 100, 100);
00013 }
```

9.2 Bossturret.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class Bossturret : CharacterBody2D
00008 {
00012     private bool hasWeapon = true;
00013
00017     private bool counter = false;
00018
00022     private double firerate = 0.7;
00023
00027     private bool seesPlayer = false;
00028
00032     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00033
00037     private bool canFire = true;
00038
00039
00043     public override void _PhysicsProcess(double delta){
00044         if(!GetNode<BossHealth>("Health").health._alive){
00045             hasWeapon = false;
00046             if (!counter)
00047             {
00048                 GetParent().GetNode<Player>("Player").IncreaseBossCount();
00049                 counter = true;
00050             }
00051             GetNode<Sprite2D>("Turret").Visible = false;
00052         }
00053
00054         seesPlayer = CheckLineOfSight();
00055         if (hasWeapon)
00056         {
00057             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00058             if (seesPlayer && canFire)
00059             {
00060                 Fire1();
00061                 Fire2();
00062             }
00063         }
00064     }
00065
00069     public async void Fire1()
00070     {
00071         GetNode<AudioStreamPlayer>("BulletSound").Play();
00072         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00073         GetParent().AddChild(bulletInstance);
00074         bulletInstance.Position = GetNode<Node2D>("BulletPoint1").GlobalPosition;
00075         bulletInstance.RotationDegrees = RotationDegrees;
00076         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00077         canFire = false;
00078         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00079         canFire = true;
00080     }
00081
00085     public async void Fire2()
00086     {
00087         await ToSignal(GetTree().CreateTimer(0.35), "timeout");
00088         GetNode<AudioStreamPlayer>("BulletSound").Play();
00089         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00090         GetParent().AddChild(bulletInstance);
00091         bulletInstance.Position = GetNode<Node2D>("BulletPoint2").GlobalPosition;
00092         bulletInstance.RotationDegrees = RotationDegrees;
00093         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00094     }
00095
00100     public bool CheckLineOfSight()
00101     {
00102         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00103         double distance = Mathf.Sqrt((playerPos.X - Position.X) * (playerPos.X - Position.X) +
00104                                     (playerPos.Y - Position.Y) * (playerPos.Y - Position.Y));
00105         if (distance < 300)
00106         {
00107             return true;
00108         }
00109         return false;
00110     }
00111 }

```

9.3 Player.cs

```

00001 using Godot;
00002 using System;

```

```

00003
00007 public partial class Player : CharacterBody2D
00008 {
00009     #region Fields
00010
00014     private float moveSpeed = 175;
00015
00019     private bool gotKeycard = false;
00020
00024     private bool gotExtraCard = true;
00025
00029     private bool moveable = true;
00030
00034     private int bossCount = 0;
00035
00039     private InteractionArea currentInteract;
00040
00044     private AnimationTree animationTree;
00045
00049     private AnimationNodeStateMachinePlayback stateMachine;
00050
00054     private double firerate = 0.5;
00055
00059     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00060
00064     private bool canFire = true;
00065
00066     #endregion
00067
00071     public override void _Ready()
00072     {
00073         moveable = true;
00074         this.gotKeycard = false;
00075         animationTree = GetNode<AnimationTree>("AnimationTree");
00076         stateMachine = (AnimationNodeStateMachinePlayback)animationTree.Get("parameters/playback");
00077     }
00078
00079
00084     public override void _PhysicsProcess(double delta)
00085     {
00086         // Update health bars based on PlayerHealth node
00087         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Health").Value =
GetNode<PlayerHealth>("Health").health._hp;
00088         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Shield").Value =
GetNode<PlayerHealth>("Health").health._shield;
00089
00090         // Check if player is dead and update visuals accordingly
00091         if (!GetNode<PlayerHealth>("Health").health._alive)
00092         {
00093             moveable = false;
00094             GetParent().GetNode<CanvasLayer>("HUD").GetNode<NinePatchRect>("Defeat").Visible = true;
00095         }
00096
00097         // Get input directions
00098         Vector2 inputDirection = new Vector2(
00099             Input.GetActionRawStrength("right") - Input.GetActionRawStrength("left"),
00100             Input.GetActionRawStrength("down") - Input.GetActionRawStrength("up")
00101         );
00102
00103         if (moveable)
00104         {
00105             // Look at the mouse position
00106             LookAt(GetGlobalMousePosition());
00107
00108             // Fire bullet if pressed and able to fire
00109             if (Input.IsActionPressed("fire") && canFire)
00110             {
00111                 Fire();
00112             }
00113
00114             // Update velocity and move character
00115             this.Velocity = inputDirection * moveSpeed;
00116             MoveAndSlide();
00117
00118             UpdateAnimation();
00119         }
00120
00121         // Execute interaction if needed
00122         if (Input.IsActionPressed("interact"))
00123         {
00124             if (GetNode<PlayerHealth>("Health").health._alive)
00125             {
00126                 ExecuteInteract();
00127             }
00128             else
00129             {
00130                 GetTree().ChangeSceneToFile("res://levels/level_1.tscn");

```

```

00131         }
00132     }
00133
00134     // Handle other input actions
00135     if (Input.IsActionPressed("exit"))
00136     {
00137         GetTree().Quit();
00138     }
00139
00140     if (Input.IsActionPressed("cheat"))
00141     {
00142         GetTree().ChangeSceneToFile("res://levels/level_6.tscn");
00143     }
00144
00145     if (Input.IsActionPressed("speed"))
00146     {
00147         moveSpeed = 500;
00148     }
00149
00150     // Check for endgame condition
00151     if (bossCount == 2)
00152     {
00153         EndGame();
00154     }
00155 }
00156
00160 public void ExecuteInteract()
00161 {
00162     if (currentInteract != null)
00163     {
00164         // Check which type of interaction is executed
00165         switch (currentInteract.GetInteractType())
00166         {
00167             case "collect":
00168                 // Check if a keycard or a weapon is picked up
00169                 if (currentInteract.GetValue() == "keycard")
00170                 {
00171                     this.gotKeycard = true;
00172                     GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D").Visible
= true;
00173
00174                     // Update interaction labels and visibility of keycard
00175                     GetParent().GetNode<StaticBody2D>("keycard").Visible = false;
00176
00177                     GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00178
00179                     GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00180
00181                     // Change label of door
00182                     if (gotExtraCard)
00183                     {
00184                         GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
next level");
00185                     }
00186                 }
00187             else if (currentInteract.GetValue() == "keycard2")
00188             {
00189                 this.gotExtraCard = true;
00190                 GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D2").Visible
= true;
00191
00192                 // Update interaction labels and visibility of keycard
00193                 GetParent().GetNode<StaticBody2D>("keycard2").Visible = false;
00194
00195                 GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00196
00197                 GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00198
00199                 // Change label of door
00200                 if (gotKeycard)
00201                 {
00202                     GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
next level");
00203                 }
00204             }
00205             break;
00206         case "next_level":
00207             if (gotKeycard && gotExtraCard)
00208             {
00209                 GetTree().ChangeSceneToFile("res://levels/" +
GetParent().GetNode<InteractionArea>("NextLevel").GetValue());
00210             }
00211             break;
00212         case "dialogue":
00213             moveable = true;
00214             currentInteract.EndDialogue();
00215             break;

```

```

00212     }
00213     }
00214 }
00215
00219 public void UpdateAnimation()
00220 {
00221     if (this.Velocity != Vector2.Zero)
00222     {
00223         stateMachine.Travel("walk");
00224     }
00225     else
00226     {
00227         stateMachine.Travel("idle");
00228     }
00229 }
00230
00235 private void _on_interaction_area_area_entered(InteractionArea area)
00236 {
00237     if (area.GetInteractType() != "heal")
00238     {
00239         this.currentInteract = area;
00240         this.currentInteract.GetNode<Label>("Label").Text = currentInteract.GetLabel();
00241         if (area.GetInteractType() == "dialogue")
00242         {
00243             area.StartDialogue(area.GetValue());
00244             moveable = false;
00245         }
00246     }
00247     else
00248     {
00249         GetNode<PlayerHealth>("Health").health.HealBoth(500);
00250         area.GetParent().QueueFree();
00251     }
00252 }
00253
00258 private void _on_interaction_area_area_exited(InteractionArea area)
00259 {
00260     area.GetNode<Label>("Label").Text = "";
00261     this.currentInteract = null;
00262 }
00263
00267 public async void Fire()
00268 {
00269     GetNode<AudioStreamPlayer>("BulletSound").Play();
00270
00271     RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00272     GetParent().AddChild(bulletInstance);
00273
00274     bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00275     bulletInstance.RotationDegrees = RotationDegrees;
00276     bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00277
00278     canFire = false;
00279     await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00280     canFire = true;
00281 }
00282
00287 public Vector2 GetPosition()
00288 {
00289     return Position;
00290 }
00291
00296 public void SetMoveable(bool val)
00297 {
00298     moveable = val;
00299 }
00300
00305 public void SetExtraCard(bool val)
00306 {
00307     gotExtraCard = val;
00308 }
00309
00313 public void IncreaseBossCount()
00314 {
00315     bossCount++;
00316 }
00317
00321 public async void EndGame()
00322 {
00323     await ToSignal(GetTree().CreateTimer(10), "timeout");
00324     GetTree().ChangeSceneToFile("res://levels/level_7.tscn");
00325 }
00326 }

```

9.4 PlayerHealth.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class PlayerHealth : Node2D
00008 {
00012     public Health health = new Health(1000, 1000, 100, 100, 100, 100);
00013 }

```

9.5 turret.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class Turret : CharacterBody2D
00008 {
00012     private bool hasWeapon = true;
00013
00017     private double firerate = 0.65;
00018
00022     private bool seesPlayer = false;
00023
00027     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00028
00032     private bool canFire = true;
00033
00037     public override void _Ready()
00038     {
00039     }
00040
00045     public override void _PhysicsProcess(double delta)
00046     {
00047         if (!GetNode<TurretHealth>("Health").health._alive)
00048         {
00049             hasWeapon = false;
00050             GetNode<Sprite2D>("Turret").Visible = false;
00051         }
00052
00053         seesPlayer = CheckLineOfSight();
00054
00055         if (hasWeapon)
00056         {
00057             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00058
00059             if (seesPlayer && canFire)
00060             {
00061                 Fire();
00062             }
00063         }
00064     }
00065
00069     public async void Fire()
00070     {
00071         GetNode<AudioStreamPlayer>("BulletSound").Play();
00072
00073         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00074         GetParent().AddChild(bulletInstance);
00075         bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00076         bulletInstance.RotationDegrees = RotationDegrees;
00077         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00078
00079         canFire = false;
00080         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00081         canFire = true;
00082     }
00083
00088     public bool CheckLineOfSight()
00089     {
00090         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00091         double distance = Mathf.Sqrt((playerPos.X - Position.X) * (playerPos.X - Position.X) +
00092                                     (playerPos.Y - Position.Y) * (playerPos.Y - Position.Y));
00093
00094         if (distance < 300)
00095         {
00096             return true;
00097         }
00098
00099         return false;
00100     }
00101 }

```

9.6 TurretHealth.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class TurretHealth : Node2D
00008 {
00012     public Health health = new Health(100,100,100,100,100,100);
00013 }

```

9.7 GlobalUsings.cs

```

00001 global using Xunit;

```

9.8 Health.cs

```

00001 using Xunit.Sdk;
00002
00003 public class Health
00004 {
00005     public int _hp { get; private set; }
00006     public int _hpMax { get; private set; }
00007     public int _shield { get; private set; }
00008     public int _shieldMax { get; private set; }
00009     public int _resistancePhysical { get; private set; }
00010     public int _resistanceElectric { get; private set; }
00011     public int _vulnPhysical { get; private set; }
00012     public int _vulnElectric { get; private set; }
00013     public bool _alive { get; private set; } = true;
00014     public const int _damageBias = 12;
00015
00016     public Health(int hpMax, int shieldMax, int initialResistancePhysical, int
initialResistanceElectric, int vulnPhysical, int vulnElectric)
00017     {
00018         _hpMax = hpMax;
00019         _shieldMax = shieldMax;
00020         _hp = _hpMax;
00021         _shield = _hpMax;
00022         _resistancePhysical = initialResistancePhysical;
00023         _resistanceElectric = initialResistanceElectric;
00024         _vulnElectric = vulnElectric;
00025         _vulnPhysical = vulnPhysical;
00026     }
00027
00028     public void ApplyTrueDamage(int damage)
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;
00033
00034         if (_hp <= 0)
00035         {
00036             _hp = 0;
00037             _alive = false;
00038         }
00039     }
00040     public void ApplyDamage(int damagePhysical, int damageElectric)
00041     {
00042         // line-by-line for easy debugging
00043         // doing stuff just twice doesn't warrant its own function
00044
00045         int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046         int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048         int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049         int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051         int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053         ApplyTrueDamage(damageTotal);
00054     }
00055
00056     public void HealHP(int amount)
00057     {
00058         _hp = Math.Min(_hp + amount, _hpMax);
00059     }
00060
00061     public void HealShield(int amount)
00062     {
00063         _shield = Math.Min(_shield + amount, _shieldMax);
00064     }

```

```

00065
00066     public void HealBoth(int healAmount)
00067     {
00068         int healthMissing = _hpMax - _hp;
00069
00070         _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072         if (healAmount > healthMissing)
00073         {
00074             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075         }
00076     }
00077
00078     public void DestroyShield()
00079     {
00080         _shield = 0;
00081     }
00082     public void RestoreHealth()
00083     {
00084         _hp = _hpMax;
00085     }
00086
00087     public void RestoreShield()
00088     {
00089         _shield = _shieldMax;
00090     }
00091
00092     public void damageShield(int damage)
00093     {
00094         _shield = Math.Max(_shield - damage, 0);
00095     }
00096
00097     public void damageHP(int damage)
00098     {
00099         _hp = Math.Max(_hp - damage, 0);
00100         _alive = Convert.ToBoolean(_hp);
00101     }
00102
00103     public void Revive()
00104     {
00105         _hp = 1;
00106         _shield = 0;
00107         _alive = true;
00108     }
00109 }

```

9.9 Health.cs

```

00001 using Xunit.Sdk;
00002
00006 public class Health
00007 {
00011     public int _hp { get; private set; }
00012
00016     public int _hpMax { get; private set; }
00017
00021     public int _shield { get; private set; }
00022
00026     public int _shieldMax { get; private set; }
00027
00031     public int _resistancePhysical { get; private set; }
00032
00036     public int _resistanceElectric { get; private set; }
00037
00041     public int _vulnPhysical { get; private set; }
00042
00046     public int _vulnElectric { get; private set; }
00047
00051     public bool _alive { get; private set; } = true;
00052
00056     public const int _damageBias = 12;
00057
00067     public Health(int hpMax, int shieldMax, int initialResistancePhysical, int
initialResistanceElectric, int vulnPhysical, int vulnElectric)
00068     {
00069         _hpMax = hpMax;
00070         _shieldMax = shieldMax;
00071         _hp = _hpMax;
00072         _shield = _hpMax;
00073         _resistancePhysical = initialResistancePhysical;
00074         _resistanceElectric = initialResistanceElectric;
00075         _vulnElectric = vulnElectric;
00076         _vulnPhysical = vulnPhysical;
00077     }

```

```
00078
00083 public void ApplyTrueDamage(int damage)
00084 {
00085     int damageAfterShield = Math.Max(damage - _shield, 0);
00086     _shield = Math.Max(_shield - damage, 0);
00087     _hp = _hp - damageAfterShield;
00088
00089     if (_hp <= 0)
00090     {
00091         _hp = 0;
00092         _alive = false;
00093     }
00094 }
00095
00101 public void ApplyDamage(int damagePhysical, int damageElectric)
00102 {
00103     // line-by-line for easy debugging
00104     // doing stuff just twice doesn't warrant its own function
00105
00106     int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00107     int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00108
00109     int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00110     int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00111
00112     int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00113
00114     ApplyTrueDamage(damageTotal);
00115 }
00116
00121 public void HealHP(int amount)
00122 {
00123     _hp = Math.Min(_hp + amount, _hpMax);
00124 }
00125
00130 public void HealShield(int amount)
00131 {
00132     _shield = Math.Min(_shield + amount, _shieldMax);
00133 }
00134
00139 public void HealBoth(int healAmount)
00140 {
00141     int healthMissing = _hpMax - _hp;
00142
00143     _hp = Math.Min(_hp + healAmount, _hpMax);
00144
00145     if (healAmount > healthMissing)
00146     {
00147         _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00148     }
00149 }
00150
00154 public void DestroyShield()
00155 {
00156     _shield = 0;
00157 }
00158
00162 public void RestoreHealth()
00163 {
00164     _hp = _hpMax;
00165 }
00166
00170 public void RestoreShield()
00171 {
00172     _shield = _shieldMax;
00173 }
00174
00179 public void damageShield(int damage)
00180 {
00181     _shield = Math.Max(_shield - damage, 0);
00182 }
00183
00188 public void damageHP(int damage)
00189 {
00190     _hp = Math.Max(_hp - damage, 0);
00191     _alive = Convert.ToBoolean(_hp);
00192 }
00193
00197 public void Revive()
00198 {
00199     _hp = 1;
00200     _shield = 0;
00201     _alive = true;
00202 }
00203 }
```

9.10 UnitTest1.cs

```

00001 namespace TestProject1;
00002 using Xunit;
00003 using Xunit.Abstractions;
00004
00005 public class UnitTest1
00006 {
00007     private readonly ITestOutputHelper _testLogger;
00008     public UnitTest1(ITestOutputHelper testLogger)
00009     {
00010         this._testLogger = testLogger;
00011     }
00012
00013     [Fact]
00014     public void Health_Initialization_Test()
00015     {
00016         int initialResistancePhysical = 100;
00017         int initialResistanceElectric = 300;
00018         int initialVulnPhysical = 0;
00019         int initialVulnElectric = 200;
00020
00021         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00022             initialVulnPhysical, initialVulnElectric);
00023
00024         Assert.Equal(1000, health._hpMax);
00025         Assert.Equal(1000, health._hp);
00026         Assert.Equal(1000, health._shield);
00027         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00028         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00029         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00030         Assert.Equal(initialVulnElectric, health._vulnElectric);
00031         Assert.True(health._alive);
00032     }
00033
00034     [Theory]
00035     [InlineData(1000, 1000, 200, 1000, 800, true)] //Damage only to Shield
00036     [InlineData(1000, 1000, 1000, 1000, 0, true)] //Exactly destroy shield
00037     [InlineData(1000, 1000, 1000, 1200, 800, 0, true)] //Destroy shield with extra damage to health
00038     [InlineData(1000, 1000, 3000, 0, 0, false)] //Overkill
00039     [InlineData(1000, 1000, 2000, 0, 0, false)] //Exact deadly damage
00040     [InlineData(1000, 1000, 0, 1000, 1000, true)] //No damage
00041     public void True_Damage_Test(
00042         int startHP, int startShield, int appliedDamage,
00043         int expectedHp, int expectedShield, bool expectedAlive)
00044     {
00045         // Arrange
00046         Health health = new Health(startHP, startShield, 1000, 1000, 0, 0);
00047
00048         // Act
00049         health.ApplyTrueDamage(appliedDamage);
00050
00051         // Assert
00052         Assert.Equal(expectedHp, health._hp);
00053         Assert.Equal(expectedShield, health._shield);
00054         Assert.Equal(expectedAlive, health._alive);
00055     }
00056
00057     [Theory]
00058     [InlineData(100, 100, 0, 0, 9000, 9000, 0, 0, false)] //Overkill
00059     [InlineData(100, 100, 100, 100, 300, 300, 1000, 406, true)] //Calculation Test + no shield break
00060     [InlineData(500, 500, 1000, 1000, 500, 500, 1000, 0, true)] //Exactly destroy shield
00061     [InlineData(500, 500, 1000, 1000, 750, 750, 500, 0, true)] //Destroy shield with extra damage to
health
00062     [InlineData(500, 500, 1000, 1000, 1000, 1000, 0, 0, false)] //Exactly deadly damage
00063     [InlineData(1000, 1000, 1000, 1000, 9000, 9000, 1000, 1000, true)] //Resistance blocks damage
00064     public void ApplyDamage_Test(
00065         int initialResistancePhysical, int initialResistanceElectric,
00066         int initialVulnPhysical, int initialVulnElectric,
00067         int damagePhysical, int damageElectric,
00068         int expectedHp, int expectedShield, bool expectedAlive)
00069     {
00070         // Arrange
00071         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00072             initialVulnPhysical, initialVulnElectric);
00073
00074         // Act
00075         health.ApplyDamage(damagePhysical, damageElectric);
00076
00077         // Assert
00078         Assert.Equal(expectedHp, health._hp);
00079         Assert.Equal(expectedShield, health._shield);
00080         Assert.Equal(expectedAlive, health._alive);
00081     }
00082
00083     [Theory]
00084     [InlineData(1000, 1000, 0, 0, 0, 1000, 1000, true)] // heal nothing

```

```

00085     [InlineData(1000, 1000, 500, 0, 250, 750, 1000, true)] // heal HP only
00086     [InlineData(1000, 1000, 500, 0, 1000, 1000, 1000, true)] // overheal HP only
00087     [InlineData(1000, 1000, 0, 500, 250, 1000, 750, true)] // heal Shield only
00088     [InlineData(1000, 1000, 0, 500, 1000, 1000, 1000, true)] // overheal Shield only
00089     [InlineData(1000, 1000, 500, 500, 750, 1000, 750, true)] // partial heal both
00090     [InlineData(1000, 1000, 500, 500, 1000, 1000, 1000, true)] // heal both
00091     [InlineData(1000, 1000, 500, 500, 2000, 1000, 1000, true)] // overheal both
00092     public void HealBoth_Test(
00093         int initialHP, int initialShield,
00094         int damageHP, int damageShield, int healAmount,
00095         int expectedHp, int expectedShield, bool expectedAlive)
00096     {
00097         // Arrange
00098         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00099
00100         // Act
00101         health.damageHP(damageHP);
00102         health.damageShield(damageShield);
00103
00104         health.HealBoth(healAmount);
00105
00106         // Assert
00107         Assert.Equal(expectedHp, health._hp);
00108         Assert.Equal(expectedShield, health._shield);
00109         Assert.Equal(expectedAlive, health._alive);
00110     }
00111 }

```

9.11 UnitTest1.cs

```

00001 namespace TestProject1;
00002 using Xunit;
00003 using Xunit.Abstractions;
00004
00008 public class UnitTest1
00009 {
00010     private readonly ITestOutputHelper _testLogger;
00011     public UnitTest1(ITestOutputHelper testLogger)
00012     {
00013         this._testLogger = testLogger;
00014     }
00015
00016     [Fact]
00020     public void Health_Initialization_Test()
00021     {
00022         int initialResistancePhysical = 100;
00023         int initialResistanceElectric = 300;
00024         int initialVulnPhysical = 0;
00025         int initialVulnElectric = 200;
00026
00027         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00028             initialVulnPhysical, initialVulnElectric);
00029
00030         Assert.Equal(1000, health._hpMax);
00031         Assert.Equal(1000, health._hp);
00032         Assert.Equal(1000, health._shield);
00033         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00034         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00035         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00036         Assert.Equal(initialVulnElectric, health._vulnElectric);
00037         Assert.True(health._alive);
00038     }
00039
00040     [Theory]
00041     [InlineData(1000, 1000, 200, 1000, 800, true)] // Damage only to Shield
00042     [InlineData(1000, 1000, 999, 1000, 1, true)] // Shield - 1
00043     [InlineData(1000, 1000, 1000, 1000, 0, true)] // Exactly destroy shield
00044     [InlineData(1000, 1000, 1001, 999, 0, true)] // Shield + 1
00045     [InlineData(1000, 1000, 1200, 800, 0, true)] // Destroy shield with extra damage to health
00046     [InlineData(1000, 1000, 1999, 1, 0, true)] // 1hp
00047     [InlineData(1000, 1000, 2000, 0, 0, false)] // Exact deadly damage
00048     [InlineData(1000, 1000, 3000, 0, 0, false)] // Overkill
00049     [InlineData(1000, 1000, 0, 1000, 1000, true)] // No damage
00053     public void True_Damage_Test(
00054         int startHP, int startShield, int appliedDamage,
00055         int expectedHp, int expectedShield, bool expectedAlive)
00056     {
00057         // Arrange
00058         Health health = new Health(startHP, startShield, 1000, 1000, 0, 0);
00059
00060         // Act
00061         health.ApplyTrueDamage(appliedDamage);
00062
00063         // Assert

```

```

00064         Assert.Equal(expectedHp, health._hp);
00065         Assert.Equal(expectedShield, health._shield);
00066         Assert.Equal(expectedAlive, health._alive);
00067     }
00068
00069     [Theory]
00070     [InlineData(100, 100, 0, 0, 9000, 9000, 0, 0, false)] // Overkill
00071     [InlineData(100, 100, 100, 100, 300, 300, 1000, 406, true)] // Calculation Test + no shield break
00072     [InlineData(500, 500, 1000, 1000, 500, 500, 1000, 0, true)] // Exactly destroy shield
00073     [InlineData(500, 500, 1000, 1000, 750, 750, 500, 0, true)] // Destroy shield with extra damage to
health
00074     [InlineData(500, 500, 1000, 1000, 1000, 1000, 0, 0, false)] // Exactly deadly damage
00075     [InlineData(1000, 1000, 1000, 1000, 9000, 9000, 1000, 1000, true)] // Resistance blocks damage
00079     public void ApplyDamage_Test(
00080         int initialResistancePhysical, int initialResistanceElectric,
00081         int initialVulnPhysical, int initialVulnElectric,
00082         int damagePhysical, int damageElectric,
00083         int expectedHp, int expectedShield, bool expectedAlive)
00084     {
00085         // Arrange
00086         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00087             initialVulnPhysical, initialVulnElectric);
00088
00089         // Act
00090         health.ApplyDamage(damagePhysical, damageElectric);
00091
00092         // Assert
00093         Assert.Equal(expectedHp, health._hp);
00094         Assert.Equal(expectedShield, health._shield);
00095         Assert.Equal(expectedAlive, health._alive);
00096     }
00097
00098     [Theory]
00099     [InlineData(1000, 1000, 0, 0, 0, 1000, 1000, true)] // heal nothing
00100     [InlineData(1000, 1000, 500, 0, 250, 750, 1000, true)] // heal HP only
00101     [InlineData(1000, 1000, 500, 0, 1000, 1000, 1000, true)] // overheal HP only
00102     [InlineData(1000, 1000, 0, 500, 250, 1000, 750, true)] // heal Shield only
00103     [InlineData(1000, 1000, 0, 500, 1000, 1000, 1000, true)] // overheal Shield only
00104     [InlineData(1000, 1000, 500, 500, 750, 1000, 750, true)] // partial heal both
00105     [InlineData(1000, 1000, 500, 500, 1000, 1000, 1000, true)] // heal both
00106     [InlineData(1000, 1000, 500, 500, 2000, 1000, 1000, true)] // overheal both
00110     public void HealBoth_Test(
00111         int initialHP, int initialShield,
00112         int damageHP, int damageShield, int healAmount,
00113         int expectedHp, int expectedShield, bool expectedAlive)
00114     {
00115         // Arrange
00116         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00117
00118         // Act
00119         health.damageHP(damageHP);
00120         health.damageShield(damageShield);
00121
00122         health.HealBoth(healAmount);
00123
00124         // Assert
00125         Assert.Equal(expectedHp, health._hp);
00126         Assert.Equal(expectedShield, health._shield);
00127         Assert.Equal(expectedAlive, health._alive);
00128     }
00129 }

```

9.12 InteractionArea.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class InteractionArea : Area2D
00008 {
00012     [Export]
00013     private String interactionLabel = "none";
00014
00018     [Export]
00019     private String interactionValue = "none";
00020
00024     [Export]
00025     private String interactionType = "none";
00026
00031     public String GetLabel()
00032     {
00033         return interactionLabel;
00034     }
00035
00040     public String GetInteractType()

```

```

00041     {
00042         return interactionType;
00043     }
00044
00049     public String GetValue()
00050     {
00051         return interactionValue;
00052     }
00053
00058     public void SetLabel(String newLabel)
00059     {
00060         interactionLabel = newLabel;
00061     }
00062
00067     public void SetInteractType(String newType)
00068     {
00069         interactionType = newType;
00070     }
00071
00076     public void SetValue(String newValue)
00077     {
00078         interactionValue = newValue;
00079     }
00080
00084     public override void _Ready()
00085     {
00086         GetNode<NinePatchRect>("DialogueManager").Visible = false;
00087     }
00088
00093     public void StartDialogue(String dialogueNumber)
00094     {
00095         GetNode<NinePatchRect>("DialogueManager").Visible = true;
00096
00097         switch (dialogueNumber)
00098         {
00099             case "0":
00100                 DisplayDialogue("Info!", "Welcome to Heidelberg Dynamics! You are L41k4, and need to
escape this facility." +
00101                 " Use [W][A][S][D] to move to the next room. Press [E] to
continue.");
00102                 break;
00103             case "1":
00104                 DisplayDialogue("Info!", "You need to be careful. This facility is full of enemie
turrets that will" +
00105                 " shoot you on sight. Use [LMB] to shoot your weapon.");
00106                 break;
00107             case "2":
00108                 DisplayDialogue("Info!", "You need to collect a keycard to open the door to the next
level.");
00109                 break;
00110             case "3":
00111                 DisplayDialogue("Info!", "You escaped! You can now enjoy your freedom.");
00112                 break;
00113             default:
00114                 EndDialogue();
00115                 break;
00116         }
00117     }
00118
00124     public void DisplayDialogue(String name, String text)
00125     {
00126         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Name").Text = name;
00127         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Text").Text = text;
00128     }
00129
00133     public void EndDialogue()
00134     {
00135         GetNode<NinePatchRect>("DialogueManager").Visible = false;
00136         QueueFree();
00137     }
00138 }

```

9.13 level_4.cs

```

00001 using Godot;
00002 using System;
00003
00007 public partial class Level_4 : Node2D
00008 {
00012     public override void _Ready()
00013     {
00014         // Reset the player's extra card state to false
00015         GetNode<Player>("Player").SetExtraCard(false);
00016

```

```

00017         // Set the interaction value of the keycard2 to "keycard2"
00018
00019         GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetValue("keycard2");
00020     }
00021 }

```

9.14 explosion.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class explosion : AnimatedSprite2D
00005 {
00006     public override void _Ready() {
00007         Position = new Vector2(0,0);
00008         //Position = GetParent().GlobalPosition;
00009     }
00010
00011     private void _on_animation_finished() {
00012         QueueFree();
00013     }
00014 }

```

9.15 projectile.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class Projectile : RigidBody2D
00005 {
00006     private PackedScene explosion = GD.Load<PackedScene>("res://projectiles/explosion.tscn");
00007
00008     private Timer timer;
00009
00010     public override void _Ready()
00011     {
00012         // Create a timer to destroy the projectile after a set duration.
00013         timer = new Timer();
00014         this.AddChild(timer);
00015         timer.WaitTime = 0.34; // Adjust the lifespan as needed
00016         timer.OneShot = true;
00017         timer.Start();
00018     }
00019
00020     private void _on_body_entered(Node body)
00021     {
00022         // Check if the collision is with a valid target
00023         if (!body.IsInGroup("entity"))
00024         {
00025             // If not, destroy the projectile
00026             QueueFree();
00027         }
00028         else if (body.IsInGroup("player"))
00029         {
00030             // If it's the player, damage them and play a hit sound
00031             QueueFree();
00032             body.GetNode<PlayerHealth>("Health").health.ApplyDamage(50, 50);
00033             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00034         }
00035         else if (body.IsInGroup("boss"))
00036         {
00037             // If it's a boss, damage them and play a hit sound
00038             QueueFree();
00039             body.GetNode<BossHealth>("Health").health.ApplyDamage(50, 50);
00040             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00041         }
00042         else
00043         {
00044             // If it's a turret, damage it and play a hit sound
00045             QueueFree();
00046             body.GetNode<TurretHealth>("Health").health.ApplyDamage(50, 50);
00047             body.GetNode<AudioStreamPlayer>("HitSound").Play();
00048         }
00049     }
00050
00051     public override void _PhysicsProcess(double delta)
00052     {
00053         // Destroy the projectile if the timer has expired
00054         if (timer.IsStopped())
00055         {
00056             QueueFree();
00057         }
00058     }
00059 }

```

```
00077     }  
00078     }  
00079 }
```


Index

- _PhysicsProcess
 - Bossturret, [11](#)
 - Player, [38](#)
 - Projectile, [49](#)
 - Turret, [53](#)
 - _Ready
 - explosion, [15](#)
 - InteractionArea, [29](#)
 - Level_4, [34](#)
 - Player, [39](#)
 - Projectile, [49](#)
 - Turret, [53](#)
 - _alive
 - Health, [25](#)
 - _damageBias
 - Health, [24](#)
 - _hp
 - Health, [25](#)
 - _hpMax
 - Health, [25](#)
 - _on_animation_finished
 - explosion, [15](#)
 - _on_body_entered
 - Projectile, [49](#)
 - _on_interaction_area_area_entered
 - Player, [38](#)
 - _on_interaction_area_area_exited
 - Player, [38](#)
 - _resistanceElectric
 - Health, [25](#)
 - _resistancePhysical
 - Health, [25](#)
 - _shield
 - Health, [25](#)
 - _shieldMax
 - Health, [26](#)
 - _testLogger
 - TestProject1.UnitTest1, [63](#)
 - _vulnElectric
 - Health, [26](#)
 - _vulnPhysical
 - Health, [26](#)
- animationTree
 - Player, [42](#)
- ApplyDamage
 - Health, [19](#)
- ApplyDamage_Test
 - TestProject1.UnitTest1, [60](#)
- ApplyTrueDamage
 - Health, [19](#), [20](#)
- bossCount
 - Player, [42](#)
- BossHealth, [7](#)
 - health, [9](#)
- Bossturret, [9](#)
 - _PhysicsProcess, [11](#)
 - bullet, [12](#)
 - canFire, [12](#)
 - CheckLineOfSight, [11](#)
 - counter, [12](#)
 - Fire1, [11](#)
 - Fire2, [12](#)
 - firerate, [13](#)
 - hasWeapon, [13](#)
 - seesPlayer, [13](#)
- bullet
 - Bossturret, [12](#)
 - Player, [43](#)
 - Turret, [54](#)
- canFire
 - Bossturret, [12](#)
 - Player, [43](#)
 - Turret, [54](#)
- characters/BossHealth.cs, [63](#)
- characters/Bossturret.cs, [64](#)
- characters/Player.cs, [64](#)
- characters/PlayerHealth.cs, [68](#)
- characters/turret.cs, [68](#)
- characters/TurretHealth.cs, [69](#)
- CheckLineOfSight
 - Bossturret, [11](#)
 - Turret, [53](#)
- counter
 - Bossturret, [12](#)
- currentInteract
 - Player, [43](#)
- damageHP
 - Health, [20](#)
- damageShield
 - Health, [21](#)
- DestroyShield
 - Health, [21](#)
- DisplayDialogue
 - InteractionArea, [29](#)
- EndDialogue
 - InteractionArea, [29](#)
- EndGame
 - Player, [40](#)
- ExecuteInteract
 - Player, [40](#)
- explosion, [14](#)
 - _Ready, [15](#)
 - _on_animation_finished, [15](#)
 - Projectile, [50](#)
- Fire
 - Player, [41](#)

- Turret, 54
- Fire1
 - Bossturret, 11
- Fire2
 - Bossturret, 12
- firerate
 - Bossturret, 13
 - Player, 43
 - Turret, 54
- Game Systems, tested using Xunit, 1, 3
- game-systems/TestProject1/GlobalUsings.cs, 69
- game-systems/TestProject1/Health.cs, 69
- game-systems/TestProject1/UnitTest1.cs, 72
- GetInteractType
 - InteractionArea, 30
- GetLabel
 - InteractionArea, 30
- GetPosition
 - Player, 41
- GetValue
 - InteractionArea, 30
- gotExtraCard
 - Player, 43
- gotKeycard
 - Player, 43
- hasWeapon
 - Bossturret, 13
 - Turret, 55
- HealBoth
 - Health, 21, 22
- HealBoth_Test
 - TestProject1.UnitTest1, 60, 61
- HealHP
 - Health, 22
- HealShield
 - Health, 23
- Health, 15
 - _alive, 25
 - _damageBias, 24
 - _hp, 25
 - _hpMax, 25
 - _resistanceElectric, 25
 - _resistancePhysical, 25
 - _shield, 25
 - _shieldMax, 26
 - _vulnElectric, 26
 - _vulnPhysical, 26
 - ApplyDamage, 19
 - ApplyTrueDamage, 19, 20
 - damageHP, 20
 - damageShield, 21
 - DestroyShield, 21
 - HealBoth, 21, 22
 - HealHP, 22
 - HealShield, 23
 - Health, 18
 - RestoreHealth, 23
 - RestoreShield, 23, 24
 - Revive, 24
- health
 - BossHealth, 9
 - PlayerHealth, 47
 - TurretHealth, 57
- Health_Initialization_Test
 - TestProject1.UnitTest1, 61, 62
- IncreaseBossCount
 - Player, 41
- InteractionArea, 27
 - _Ready, 29
 - DisplayDialogue, 29
 - EndDialogue, 29
 - GetInteractType, 30
 - GetLabel, 30
 - GetValue, 30
 - interactionLabel, 32
 - interactionType, 32
 - interactionValue, 32
 - SetInteractType, 30
 - SetLabel, 31
 - SetValue, 31
 - StartDialogue, 31
- interactionLabel
 - InteractionArea, 32
- interactionType
 - InteractionArea, 32
- interactionValue
 - InteractionArea, 32
- Level_4, 33
 - _Ready, 34
- levels/level_4.cs, 75
- moveable
 - Player, 44
- moveSpeed
 - Player, 44
- Player, 34
 - _PhysicsProcess, 38
 - _Ready, 39
 - _on_interaction_area_area_entered, 38
 - _on_interaction_area_area_exited, 38
 - animationTree, 42
 - bossCount, 42
 - bullet, 43
 - canFire, 43
 - currentInteract, 43
 - EndGame, 40
 - ExecuteInteract, 40
 - Fire, 41
 - firerate, 43
 - GetPosition, 41
 - gotExtraCard, 43
 - gotKeycard, 43
 - IncreaseBossCount, 41

- moveable, [44](#)
- moveSpeed, [44](#)
- SetExtraCard, [41](#)
- SetMoveable, [42](#)
- stateMachine, [44](#)
- UpdateAnimation, [42](#)
- PlayerHealth, [45](#)
 - health, [47](#)
- Projectile, [47](#)
 - _PhysicsProcess, [49](#)
 - _Ready, [49](#)
 - _on_body_entered, [49](#)
 - explosion, [50](#)
 - timer, [50](#)
- projectiles/explosion.cs, [76](#)
- projectiles/projectile.cs, [76](#)
- RestoreHealth
 - Health, [23](#)
- RestoreShield
 - Health, [23](#), [24](#)
- Revive
 - Health, [24](#)
- seesPlayer
 - Bossturret, [13](#)
 - Turret, [55](#)
- SetExtraCard
 - Player, [41](#)
- SetInteractType
 - InteractionArea, [30](#)
- SetLabel
 - InteractionArea, [31](#)
- SetMoveable
 - Player, [42](#)
- SetValue
 - InteractionArea, [31](#)
- StartDialogue
 - InteractionArea, [31](#)
- stateMachine
 - Player, [44](#)
- systems/TestProject1/Health.cs, [70](#)
- systems/TestProject1/UnitTest1.cs, [73](#)
- TestProject1, [7](#)
- TestProject1.UnitTest1, [57](#)
 - _testLogger, [63](#)
 - ApplyDamage_Test, [60](#)
 - HealBoth_Test, [60](#), [61](#)
 - Health_Initialization_Test, [61](#), [62](#)
 - True_Damage_Test, [62](#)
 - UnitTest1, [59](#)
- timer
 - Projectile, [50](#)
- True_Damage_Test
 - TestProject1.UnitTest1, [62](#)
- Turret, [51](#)
 - _PhysicsProcess, [53](#)
 - _Ready, [53](#)
 - bullet, [54](#)
 - canFire, [54](#)
 - CheckLineOfSight, [53](#)
 - Fire, [54](#)
 - fireRate, [54](#)
 - hasWeapon, [55](#)
 - seesPlayer, [55](#)
- TurretHealth, [55](#)
 - health, [57](#)
- UnitTest1
 - TestProject1.UnitTest1, [59](#)
- UpdateAnimation
 - Player, [42](#)